Чернівецький Національний університет імені Юрія Федьковича
Факультет математики та інформатики

В. Косован
О. Матвій
Г. Мельник
В. Мельник

Інтелектуальний Аналіз Даних

Навчальний посібник

Чернівці, 2026

УДК 004.738.5

Рекомендовано до друку методичною радою факультету математики та інформатики
Чернівецького національного університету імені Юрія Федьковича
(протокол № від 2026 року)

Укладачі:
Василь Косован, кандидат фізико-математичних наук, асистент кафедри математичного моделювання.
Олександр Матвій, кандидат фізико-математичних наук, доцент кафедри математичного моделювання;
Галина Мельник, кандидат економічних наук, доцент кафедри прикладної математики та інформаційних технологій;
Василь Мельник, кандидат фізико-математичних наук, асистент кафедри математичного моделювання;

Інтелектуальний Аналіз Даних : навчальний посібник.. Укл.: Василь Косован, Олександр Матвій, Галина Мельник, Василь Мельник – Чернівці: Чернівецький національний університет ім. Юрія Федьковича, 2026. – с.

Посібник висвітлює базові принципи інтелектуального аналізу даних. Матеріали посібника підготовлені на основі методологічних досліджень авторів та відповідного курсу лекцій, містять основи теорії і практики аналізу даних.

Рецензенти:
Міночкін Дмитро Анатолійович, канд. щу сфері телекомунікацій, доцент кафедрителекомунікацій, Національний технічний університет України "Київський політехнічний інститут імені Ігоря Сікорського"
Семенов Віталій, сеніор інженер, SoftHouseGroup.

Для студентів, що здобувають освіту в галузі знань „Інформаційні технології” для спеціальностей 122 - “Комп’ютерні науки”, 124 - “Системний аналіз”, 113 - ”Прикладна математика” та інших спеціальностей, що пов’язані із вивченням комп’ютерних інформаційних технологій. УДК 004.738.5

© Косован В.М., Матвій О.В., Мельник Г.В., Мельник В.С.

© Чернівецький національний університет, 2026

Зміст

1 Збір даних. 6
- 1.1 Збір даних з веб-джерел. . . . 6
- 1.2 RESTful API . . . 7
- 1.3 CSV-файли. . . . 8
- 1.4 Парсинг JSON в Python. . . . 9
- 1.5 Парсинг XML/HTML в Python. . . . 10
- 1.6 Регулярні вирази. . . . 12
  - 1.6.1 Відповідність декількох потенційних символів. . . . 13
  - 1.6.2 Групування. . . . 13
  - 1.6.3 Заміни. . . . 14
  - 1.6.4 Порядок і жадібне зіставлення. . . . 14

2 Обробка даних. 17
- 2.1 Pandas . . . 17
- 2.2 Методи очищення даних та обробка відсутніх значень. . . . 20
  - 2.2.1 Видалення відсутніх даних. . . . 21
  - 2.2.2 Заповнення середнім. . . . 22
  - 2.2.3 Заміщення медіанним значенням. . . . 22
  - 2.2.4 Заміщення модою. . . . 22
- 2.3 Масштабування та нормалізація даних. . . . 23
  - 2.3.1 Мінімально-максимальне масштабування. . . . 23
  - 2.3.2 Нормалізація (стандартизація) за Z-балом. . . . 25
  - 2.3.3 Надійне масштабування. . . . 25
- 2.4 Вступ до конструювання ознак. . . . 26
  - 2.4.1 Створення нових ознак. . . . 26
  - 2.4.2 Кодування категоріальних змінних. . . . 27
  - 2.4.3 Функція взаємодії. . . . 28
  - 2.4.4 Поліноміальні ознаки. . . . 29

3 Візуалізація даних. 32
- 3.1 Типи даних. . . . 32
- 3.2 1D дані . . . 33
  - 3.2.1 Стовпчасті діаграми — категоріальні дані . . . 33
  - 3.2.2 Кругові діаграми. . . . 34
  - 3.2.3 Гістограми — числові дані . . . 35
- 3.3 2D дані . . . 36
  - 3.3.1 Точкові діаграми — числові x числові. . . . 36
  - 3.3.2 Лінійна діаграма - числові x числові. . . . 37
  - 3.3.3 Коробкова і скрипкові діаграми — категоріальні x числові . . . 38

3.3.4 Теплова карта та бульбашкові діаграми — категоріальні x категоріальні . . 40
3.4 3D дані . . . 42
3.4.1 Матриці розсіювання (точкові матриці). . . . 42
3.4.2 Бульбашкові діаграми. . . . 43

4 Лінійна алгебра та бібліотека NumPy. 47
4.1 Основи лінійної алгебри. . . . 47
4.1.1 Порядок рядків і стовпців. . . . 47
4.1.2 Основні операції. . . . 48
4.2 Масиви NumPy. . . . 48
4.2.1 Індексація в масивах NumPy. . . . 50
4.2.2 Основні операції над масивами в NumPy. . . . 51
4.2.3 Трансляція NumPy. . . . 53
4.2.4 Операції лінійної алгебри в NumPy. . . . 53
4.2.5 Складність операцій. . . . 55
4.2.6 Розріджені матриці. . . . 55

5 Обробка Природної Мови. 58
5.1 Попередня обробка тексту. . . . 58
5.1.1 Очищення тексту. . . . 58
5.1.2 Токенізація, стемінг, лематизація, нормалізація. . . . 59
5.2 Чисельне представлення тексту. . . . 61
5.2.1 Мішок слів. . . . 62
5.2.2 TFIDF та косинусна подібність тексту. . . . 62
5.2.3 Векторне подання слів. . . . 66

6 Теорія ймовірності в контексті аналізу даних. 70
6.1 Основи теорії ймовірності. . . . 70
6.1.1 Закон великих чисел . . . 71
6.2 Використання NumPy для роботи з розподілами ймовірностей. . . . 71
6.2.1 Рівномірний розподіл. . . . 71
6.2.2 Гаусівський розподіл. . . . 71
6.2.3 Дискретні розподіли. . . . 72
6.3 Теорема Байєса. . . . 73

7 Вступ в машинне навчання. 76
7.1 Градієнтний спуск. . . . 78
7.2 Трохи більше про машинне навчання. . . . 85
7.2.1 Приклади алгоритмів машинного навчання. . . . 85

8 Лінійна Класифікація. 87

9 Нелінійні Алгоритми Машинного Навчання. 88

10 Ансамблеве навчання. 89

11 Навчання без учителя. 90

12 Перевірка статистичних гіпотез. 91

13 Рекомендаційні Системи. 92

14 Часові Ряди. **93**

15 Моделювання Кризових Явищ. **94**

16 Використання нейронних мереж для аналізу даних. **95**

Розділ 1

Збір даних.

Першим етапом практично будь-якого проєкту з інтелектуального аналізу даних, машинного навчання або аналітики є збір даних (Data Collection). Якість отриманих результатів значною мірою залежить від якості та повноти вихідних даних. Навіть найскладніші алгоритми машинного навчання не здатні компенсувати відсутність важливої інформації або низьку якість даних, тому процес їх отримання є одним із найважливіших етапів підготовки аналітичного проєкту.

Сучасні дані можуть надходити з великої кількості різноманітних джерел. Це можуть бути реляційні бази даних, електронні таблиці, CSV-файли, журнали подій (logs), сенсори Інтернету речей (IoT), соціальні мережі, веб-сайти, а також численні програмні інтерфейси (API), що надають доступ до інформаційних сервісів. У деяких випадках дані вже доступні у вигляді готових файлів і можуть бути безпосередньо завантажені для подальшого аналізу. В інших ситуаціях необхідно виконувати запити до баз даних або зовнішніх інформаційних систем для отримання актуальної інформації.

Автоматизований збір даних є значно швидшим, дозволяє обробляти великі обсяги інформації, зменшує кількість людських помилок і забезпечує регулярне оновлення даних без постійного втручання користувача. Основними труднощами під час збору даних є різні формати даних, неоднакова структура, дублікати, пропущені значення, обмеження доступу, необхідність автентифікації та різна якість отриманої інформації.

В цій темі ми поговоримо про збір даних з двох основних джерел:

- запит до API (більшість з яких сьогодні є веб-орієнтованими); та
- вилучення даних з веб-сторінки за допомогою технологій веб-скрапінгу (Web Scraping).

1.1 Збір даних з веб-джерел.

Веб-джерело даних — це будь-який ресурс у мережі Інтернет, з якого можна отримати інформацію, наприклад веб-сторінка, REST API або онлайн-база даних. Для збору даних можуть використовуватися веб-сайти, REST API, XML- та JSON-сервіси, відкриті набори даних, хмарні сервіси та веб-додатки.

Сьогодні переважна більшість автоматизованих систем обміну даними використовує протокол HTTP (HyperText Transfer Protocol). Хоча спочатку HTTP був створений для передачі вебсторінок між браузерами та веб-серверами, з часом він став універсальним механізмом взаємодії між програмними системами. Саме на основі HTTP функціонує більшість сучасних веб-сервісів, REST API та хмарних платформ.

Під час збору даних клієнтська програма надсилає HTTP-запит до віддаленого сервера, а сервер повертає відповідь, яка може містити HTML-документ, JSON-структуру, XML-файл, зображення або інші типи даних. Розуміння принципів роботи HTTP є необхідною передумовою

для ефективного використання веб-джерел даних, тому в наступних розділах буде розглянуто основні типи HTTP-запитів, структуру відповідей сервера та способи обробки отриманих даних у Python.

HTTP-відповідь — це повідомлення, яке сервер надсилає клієнту після обробки HTTP-запиту. Вона містить код стану, заголовки та тіло відповіді, у якому можуть передаватися HTML, JSON, XML або інші дані.

Веб-скрапінг — це автоматизований процес отримання та вилучення інформації з HTML-сторінок веб-сайтів для її подальшого аналізу або обробки. Процес веб-скрапінгу включає отримання HTML-сторінки, аналіз її структури, пошук потрібних елементів, вилучення необхідних даних та їх подальше збереження або обробку.

Процес веб-скрапінгу складається із надсилання запиту, одержання відповіді, парсингу тексту, витягу очищення та збереження даних.

```
import requests
response = requests.get("https://fmi.chnu.edu.ua/")

print("Status Code:", response.status_code)
print("Headers:", response.headers)

print(response.text)
```

Результат:

```
Status Code: 200
Headers: {'Date': 'Fri, 12 Sep 2025 05:44:43 GMT', 'Content-Type': 'text/html; charset=utf-8' ...
```

Коди стану HTTP повідомляють клієнту про результат обробки запиту сервером. Код 200 (OK) означає, що запит успішно виконано. Код 404 (Not Found) вказує на те, що запитуваний ресурс не знайдено. Код 500 (Internal Server Error) свідчить про виникнення внутрішньої помилки на сервері під час обробки запиту.

HTTP GET є найпоширенішим методом, але існують також методи PUT, POST, DELETE, які змінюють деякі параметри на сервері.

Параметри URL можна вказати за допомогою бібліотеки запитів, як показано нижче:

```
params = {"sa":"t", "rct":"j", "q":"", "esrc":"s",
"source":"web", "cd":"9", "cad":"rja", "uact":"8"}
response = requests.get("http://www.google.com/url", params=params)

print("Final URL:", response.url)
print(response.text)
```

Результат:

```
Final URL: http://www.google.com/url?sa=t&rct=j&q=&esrc=s&source=web&cd=9&cad=rja&uact=8
<html lang="uk"><head><meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1"><title>&#1055
    ...
```

1.2 RESTful API

Якщо ви перейдете від простого запиту веб-сторінок до веб-API, ви, швидше за все, зіткнетеся з REST API (Representational State Transfer). REST — це скоріше архітектура дизайну, але ось

кілька ключових моментів:

- Використовує стандартний інтерфейс і методи HTTP (GET, PUT, POST, DELETE)
- Stateless – сервер не запам'ятовує, що ви робили
- Практичне правило: якщо ви надсилаєте ключ свого облікового запису разом із кожним викликом API, ви, ймовірно, використовуєте REST API

REST API забезпечує стандартизований, швидкий і надійний доступ до актуальних даних, дозволяє автоматизувати їх отримання та легко інтегрується з різними програмними системами. Ви запитуєте REST API подібно до стандартних HTTP-запитів, але майже завжди потрібно включати параметри. Параметри можуть передаватися як частина URL-адреси, через рядок запиту (Query Parameters), у заголовках HTTP або в тілі запиту залежно від використовуваного HTTP-методу.

Автентифікація API — це процес підтвердження особи користувача або програми перед наданням доступу до ресурсів сервера. Найпоширенішими способами є API Key, Bearer Token (JWT), Basic Authentication та OAuth 2.0.

Отримайте власний токен доступу на https://github.com/settings/tokens/new GitHub API використовує GET/PUT/DELETE, щоб ви могли автоматично запитувати або оновлювати елементи у вашому обліковому записі GitHub.

☞ Сервер не запам'ятовує ваші останні запити, ви завжди повинні включати свій токен доступу, якщо використовуєте його таким чином.

Приклад REST запиту:

```
import os
from dotenv import load_dotenv

load_dotenv()
token = os.getenv("GITHUB_TOKEN")

headers = {'Authorization': 'Bearer '+token}
response = requests.get("https://api.github.com/user", headers=headers)
print(response.content)
```

Результат:

```
b'{"login":"maxvonlancaster","id":38358502,"node_id":"MDQ6VXNlcjM4MzU4NTAy","avatar_url": ...
```

REST-архітектура базується на взаємодії клієнта і сервера, використанні стандартних HTTP-методів, відсутності збереження стану між запитами (stateless), єдиному інтерфейсі доступу до ресурсів та представленні ресурсів за допомогою URI. Дані найчастіше передаються у форматах JSON або XML.

HTTP-заголовки використовуються для передачі службової інформації між клієнтом і сервером. Вони можуть містити тип передаваних даних, параметри автентифікації, інформацію про кешування, мову, кодування, тип клієнта та інші параметри, необхідні для коректної обробки запиту або відповіді.

1.3 CSV-файли.

CSV (Comma-Separated Values): Відноситься до будь-якого текстового файлу з роздільниками (не завжди розділеного комами).

Якщо самі значення містять коми, ви можете взяти їх у лапки (наш реєстратор, очевидно, завжди так робить, щоб бути впевненим).

Імпортуємо дані із csv-файлу, що містить дані про історичну ціну біткоїна, та зобразимо ці дані, за допомогою наступного коду.

```
import pandas as pd

dataframe = pd.read_csv("../resources/bitcoin.csv", delimiter=',', quotechar='"')
dataframe.head(10)
```

Результат:

	Date	Open	High	Low	Close	Volume	Currency
0	2010-07-18	0.0	0.1	0.1	0.1	75	USD
1	2010-07-19	0.1	0.1	0.1	0.1	574	USD
2	2010-07-20	0.1	0.1	0.1	0.1	262	USD
3	2010-07-21	0.1	0.1	0.1	0.1	575	USD
4	2010-07-22	0.1	0.1	0.1	0.1	2160	USD
5	2010-07-23	0.1	0.1	0.1	0.1	2403	USD
6	2010-07-24	0.1	0.1	0.1	0.1	496	USD
7	2010-07-25	0.1	0.1	0.1	0.1	1551	USD
8	2010-07-26	0.1	0.1	0.1	0.1	877	USD
9	2010-07-27	0.1	0.1	0.1	0.1	3374	USD

Формат CSV є простим, компактним і зрозумілим, підтримується більшістю програм для роботи з даними та легко обробляється різними мовами програмування. Він забезпечує швидке зчитування й запис табличних даних, що робить його одним із найпоширеніших форматів для обміну інформацією.

☞ Формат CSV не підтримує вкладені структури даних, типи даних, метадані та форматування. Крім того, можуть виникати проблеми через використання різних роздільників, кодувань символів або символів нового рядка, що іноді ускладнює обмін даними між різними програмами.

1.4 Парсинг JSON в Python.

JSON (JavaScript Object Notation) — це легкий текстовий формат обміну даними, який використовується для передачі структурованої інформації між програмами. Незважаючи на назву, JSON сьогодні є незалежним від JavaScript і підтримується практично всіма сучасними мовами програмування.

JSON широко використовується у REST API, веб-сервісах, мобільних застосунках та хмарних системах. Основними перевагами JSON є компактність, простота читання людиною та легкість автоматичної обробки.

Вбудована бібліотека для читання/запису об'єктів Python з/у файли JSON, імпортується за допомогою команди 'import json'. Нижче наведено приклад роботи із json-даними в python. REST-запит вертає json дані, які ми потім відображаємо.

```
import json
# load json response from REST API

headers = {'Authorization': 'token ' + token}
response = requests.get("https://api.github.com/user", headers=headers)
data = json.loads(response.content)

#json.load(file) # load json from file
```

```
json.dumps(data) # return json string
#json.dump(obj, file) # write json to file
```

Результат:

```
'{"login": "maxvonlancaster", "id": 38358502, "node_id": "MDQ6VXNlcjM4MzU4NTAy", "avatar_url": ...
```

JSON підтримує рядки, числа, логічні значення, масиви, об'єкти, а також спеціальне значення 'null'. JSON-файл можна зчитати за допомогою стандартного модуля 'json', використовуючи функцію 'json.load()', або за допомогою бібліотеки 'pandas', якщо дані мають табличну структуру.

Серіалізація JSON — це процес перетворення об'єкта Python у JSON-рядок або JSON-файл для його збереження чи передачі. Десеріалізація JSON — це процес перетворення JSON-рядка або JSON-файлу у відповідні об'єкти Python.

Після десеріалізації json в python значення вкладених полів отримують шляхом послідовного звернення до вкладених словників або списків за допомогою ключів та індексів.

Нижче наведено приклад роботи з вкладеними структурами в JSON:

```
import json

json_string = '''
{
    "employee":
    {
        "name":"John",
        "department":
        {
            "name":"IT",
            "floor":4
        }
    }
}
'''

obj = json.loads(json_string)

print(obj["employee"]["department"]["name"])
```

Результат:

```
IT
```

1.5 Парсинг XML/HTML в Python.

XML (eXtensible Markup Language) — це мова розмітки, призначена для зберігання, структурування та обміну даними між різними інформаційними системами. XML-документ складається з декларації, кореневого елемента, вкладених тегів, атрибутів та текстового вмісту. XML використовує теги для опису структури даних і є більш громіздким, тоді як JSON має компактніший синтаксис, легше читається людиною та швидше обробляється програмами.

Існує ряд XML/HTML-парсерів для Python, але для науки про дані добре підходить бібліотека BeautifulSoup (спеціально призначена для вилучення даних з XML/HTML-файлів). Нижче наведено приклад завантаження html-розмітки із сайту, та код, що знаходить всі anchor-посилання всередені елементів ‘ ‘.

```
from bs4 import BeautifulSoup
import requests

response = requests.get("https://mathmod.chnu.edu.ua/")

root = BeautifulSoup(response.content)
root.find("div").findAll("a")


item = root.find(class_="flex-item")
print(item.text)


titles = root.select("div.flex-item h2")

for t in titles:
    print(t.text)


link = root.find("a")

print(link["href"])
print(link.text)


table = root.find("table")

rows = table.find_all("tr")

for row in rows:
    cells = row.find_all("td")
    print([cell.text.strip() for cell in cells])
```

В цьому коді ми спочатку: отримуємо всі посилання на веб-сторінці, потім шукаємо елемент за класом, CSS селектором, отримуємо атрибути тега та, в кінці, отримуємо всі рядки таблиці.

Результат:

```
[<a href="/cdn-cgi/l/email-protection#77141b105a1a16031f37141f190259121302590216"><span><span
    class="__cf_email__" data-cfemail="a0c3ccc78dcdc1d4c8e0c3c8ced58ec5c4d58ed5c1">[email
    protected]</span></span></a>,
 <a href="https://www.facebook.com/fmi.org.ua/" rel="noopener" target="_blank" title=".."><span
    class="iconify-icon footer-social-link-icon icon--fa-brands icon--fa-brands--facebook-f"></span></a>,
 <a href="https://www.instagram.com/m._i_.f/" rel="noopener" target="_blank" title=".."><span
    class="iconify-icon footer-social-link-icon icon--fa-brands icon--fa-brands--instagram"></span></a>,
 <a href="https://youtube.com/@MathTube_" rel="noopener" target="_blank" title="Youtube"><span
    class="iconify-icon footer-social-link-icon icon--fa-brands icon--fa-brands--youtube"></span></a>]
```

HTML-документ — це текстовий файл, який містить структуру веб-сторінки, описану за допомогою HTML-тегів. HTML є основним джерелом інформації під час веб-скрапінгу, оскільки саме в ньому містяться текст, таблиці, посилання, зображення та інші елементи веб-сторінки.

HTML-елемент за тегом можна знайти за допомогою методів `find()` або `find_all()`, вказавши назву потрібного тегу. Елементи знаходять, вказуючи значення атрибутів, наприклад "class "id"або інших, використовуючи параметри методів `find()` чи `find_all()`.

DOM (Document Object Model) — це деревоподібне представлення HTML-документа, у якому кожен HTML-елемент є окремим вузлом, доступним для аналізу та модифікації.

1.6 Регулярні вирази.

Після завантаження даних (або якщо вам потрібно створити парсер для завантаження даних іншого формату) часто доводиться шукати певні елементи в даних.

Регулярні вирази (Regular Expressions, скорочено RegEx або Regex) — це спеціальна мова шаблонів, призначена для пошуку, перевірки, виділення та заміни текстових фрагментів. Регулярні вирази широко застосовуються під час збору, очищення та попередньої обробки даних, оскільки дозволяють швидко знаходити рядки, що відповідають заданому шаблону. У задачах інтелектуального аналізу даних регулярні вирази використовуються для пошуку даних (електронна пошта, номери телефонів, дати/час), перевірки правильності введених даних, очищення тексту від службових символів, та попередньої підготовки текстів перед їх аналізом.

Наприклад, знайти перше входження рядка «data science»

```
import re
text = "This course will introduce the basics of data science"
match = re.search(r"data science", text)
print(match.start())
print(re.match(r"This", text))

match = re.match(r"data science", text)
match = re.search(r"data science", text)

all_matches = re.findall(r"a", text)
print(all_matches)
```

Тут метод `re.findall(r"a", text)` повертає всі збіги в тексті.

Результат:

```
41
<re.Match object; span=(0, 4), match='This'>
['a', 'a', 'a']
```

Під час збору та підготовки даних регулярні вирази використовуються для автоматичного вилучення структурованої інформації з текстів. Типовими прикладами є пошук електронних адрес, телефонних номерів, URL-адрес, IP-адрес, поштових індексів, дат, часу, грошових сум та інших шаблонних даних.

Крім пошуку інформації, регулярні вирази широко застосовуються для очищення текстів перед аналізом: видалення HTML-тегів, спеціальних символів, зайвих пробілів, службових позначок та інших небажаних елементів. Вони також використовуються для перевірки правильності введених даних (валідації) та підготовки текстових корпусів до подальшої обробки методами обробки природної мови (Natural Language Processing, NLP).

Завдяки простоті використання та високій швидкодії регулярні вирази є одним із базових інструментів інженера даних і спеціаліста з інтелектуального аналізу даних.

1.6.1 Відповідність декількох потенційних символів.

Справжня сила регулярних виразів полягає в можливості збігу декількох можливих послідовностей символів. Спеціальні символи в регулярних виразах: `.^$*+?{}\[]()|` (якщо ви хочете, щоб ці символи збігалися точно, вам потрібно їх екранувати: `\$`)

Відповідність наборів символів:

- Відповідність символу «a»: `a`
- Відповідність символів «a», «b» або «c»: `[abc]`
- Будь-який символ, крім «a», «b» або «c»: `[\^$abc]`
- Відповідність будь-якої цифри: `\d` (те саме, що ‘[0-9|‘)
- Відповідність будь-якого алфавітно-цифрового символу: `\w` (те саме, що `[a-zA-z0-9_]`)
- Відповідність пробілу: `\s` (те саме, що `[ \t\n\r\f\v]`)
- Відповідність будь-якого символу: ‘.‘ (включно з новою лінією з re.DOTALL)

Може збігатися з одним або декількома випадками символу (або набору символів)

Деякі поширені модифікатори:

- Відповідність символу «a» рівно один раз: ‘a‘
- Відповідність символу «a» нуль або один раз: ‘a?‘
- Відповідність символу «a» нуль або більше разів: ‘a*‘
- Відповідність символу «a» один або більше разів: ‘a+‘
- Відповідність символу «a» рівно n разів: ‘an‘

Можна поєднувати їх із збігом декількох символів:

- Відповідність усіх випадків «‘<something>‘ science», де ‘<something>‘ — це алфавітно-цифровий рядок, що містить принаймні один символ: `‘\w+\s+science‘`

1.6.2 Групування.

Часто нам потрібно отримати більше інформації, ніж просто те, чи знайшли ми збіг чи ні (наприклад, ми можемо хотіти знати, який текст збігся).

Групування: оберніть частини регулярного виразу в дужки, щоб «запам’ятати» ці частини збігу

`‘(\w+)\s([Aa]nalysis)‘`.

```
text = "This course will introduce the basics of data analysis"
match = re.search(r"(\w+)\s([Aa]nalysis)", text)
print(match.start(), match.groups())
```

Чому перед рядком стоїть «r»? Це дозволяє уникнути необхідності подвійного екранування рядків

Результат:

```
41 ('data', 'analysis')
```

1.6.3 Заміни.

Регулярні вирази надають механізм для заміни одного тексту іншим текстом.

```
better_text = re.sub(r"data analysis", r"schmada analysis", text)

better_text = re.sub(r"(\w+)\s([Aa])nalysis", r"\1 \2shnalysis", text)
print(better_text)
```

Щоб включити текст, який було збережено під час пошуку за допомогою груп, використовуйте екрановані послідовності \1, \2, ... у тексті заміни

Результат:

```
This course will introduce the basics of data ashnalysis
```

1.6.4 Порядок і жадібне зіставлення.

У регулярних виразах існує порядок операцій. ‘abc|def‘ відповідає рядкам «abc» або «def», а не «ab(c або d)ef». Ви можете обійти це, використовуючи дужки, наприклад ‘a(bc|de)f‘. Це також створює групу, використовуйте ‘a(?:bc|de)f‘, якщо ви не хочете її захоплювати.

За замовчуванням регулярні вирази намагаються захопити якомога більше тексту (жадібне співставлення, greedy matching). ‘<(.*)>‘, застосоване до ‘<a>text‘, буде відповідати всьому виразу. Якщо ви хочете захопити якомога менше тексту, використовуйте ‘<(.*?)>‘, це буде відповідати тільки терміну ‘<a>‘.

✎ **Питання для самоконтролю.**

1. Що таке збір даних (Data Collection) та яке його значення в процесі інтелектуального аналізу даних?
2. Які основні джерела даних використовуються в Data Mining?
3. Які переваги має автоматизований збір даних порівняно з ручним?
4. Які труднощі можуть виникати під час збору даних з різних джерел?
5. Що таке веб-джерело даних?
6. Які типи веб-джерел можуть використовуватися для збору даних?
7. Що таке веб-скрапінг (Web Scraping)?
8. Які етапи процесу веб-скрапінгу?
9. Які бібліотеки Python найчастіше використовуються для отримання веб-сторінок?
10. Для чого використовується бібліотека ‘requests‘ ?
11. Що таке HTTP-запит?
12. Які основні HTTP-методи використовуються при роботі з веб-ресурсами?
13. Що таке HTTP-відповідь?
14. Яке призначення кодів стану HTTP?
15. Що означають коди стану 200, 404 та 500?

16. Що таке RESTful API?
17. Які принципи лежать в основі REST-архітектури?
18. Чим REST API відрізняється від звичайної веб-сторінки?
19. Які переваги використання REST API для збору даних?
20. Як передаються параметри в REST API-запитах?
21. Що таке JSON-відповідь REST API?
22. Для чого використовуються HTTP-заголовки (headers)?
23. Що таке автентифікація API?
24. Які способи автентифікації API ви знаєте?
25. Що таке CSV-файл?
26. Які особливості структури CSV-файлів?
27. Які переваги має формат CSV?
28. Які недоліки має формат CSV?
29. Як зчитати CSV-файл у Python?
30. Для чого використовується бібліотека ‘pandas‘ при роботі з CSV?
31. Що таке JSON?
32. Які типи даних підтримує формат JSON?
33. Як зчитати JSON-файл у Python?
34. Що таке серіалізація JSON?
35. Що таке десеріалізація JSON?
36. Як отримати значення вкладених полів у JSON-структурі?
37. Що таке XML?
38. Які основні елементи XML-документа?
39. Чим XML відрізняється від JSON?
40. Як виконати парсинг XML у Python?
41. Що таке HTML-документ?
42. Яку роль HTML відіграє у веб-скрапінгу?
43. Для чого використовується бібліотека BeautifulSoup?
44. Як знайти HTML-елемент за тегом?
45. Як знайти HTML-елемент за атрибутом або класом?
46. Що таке DOM-структура веб-сторінки?
47. Що таке регулярний вираз (Regular Expression)?

48. Для яких задач використовуються регулярні вирази під час збору даних?
49. Які основні метасимволи регулярних виразів ви знаєте?
50. Як за допомогою регулярних виразів знаходити та витягувати потрібні фрагменти тексту?

Розділ 2

Обробка даних.

Обробка даних (Data Processing) — це сукупність методів і процедур, спрямованих на підготовку даних до подальшого аналізу та побудови моделей машинного навчання. Основними етапами обробки є очищення даних, усунення пропущених значень і дублікатів, виявлення викидів, перетворення типів даних, а також масштабування та нормалізація ознак. Якість виконання цих етапів безпосередньо впливає на точність, швидкість і надійність моделей машинного навчання, оскільки алгоритми навчаються саме на підготовлених даних. «Сирі» дані часто містять пропущені значення, дублікати, помилки введення, викиди, некоректні формати та неоднорідні масштаби ознак, що може призводити до погіршення якості аналізу та зниження точності прогнозування.

2.1 Pandas

Існує ряд бібліотек Python, які обробляють реляційні дані, зазвичай написані як інтерфейси до декількох різних систем управління реляційними базами даних (таких як PostgreSQL, MySQL та інші). [Примітка: слід зазначити, що програмне забезпечення, таке як PostreSQL і MySQL, правильніше називати системами управління реляційними базами даних (RDBMS), а не базами даних. База даних — це фактичні таблиці та записи, що визначають фактичну сукупність даних.

Однак у цьому курсі ми будемо працювати з реляційними даними переважно за допомогою двох бібліотек: Pandas і SQLite. Це особливо прості бібліотеки, якщо говорити про реальні бази даних: Pandas, безумовно, не є реальною системою реляційних баз даних (хоча вона надає функції, що віддзеркалюють деякі їхні можливості), тоді як SQLite є «реальною» RDBMS, але надзвичайно простою, без стандартної архітектури клієнт/сервер, яка є практично у всіх реальних виробничих базах даних. Проте для багатьох задач у галузі науки про дані вони будуть достатніми, тому ми зосередимося на них тут.

Ми вже коротко розглянули Panda, коли обговорювали збір даних, і вона виявилася однією з найкорисніших бібліотек Python для науки про дані. Як ми вже згадували вище (але будемо повторювати цей факт багато разів), Pandas — це не бібліотека реляційних баз даних, а бібліотека «даних-рам». Ви можете уявити собі фрейм даних як 2D-масив, за винятком того, що записи у фреймі даних можуть бути будь-яким типом об'єкта Python (і мати змішані типи в масиві), а рядки/стовпці можуть мати «мітки» замість цілочисельних індексів, як у стандартному масиві.

Давайте подивимося, як спочатку створити фрейм даних у Pandas, який відображає таблицю Person із записами про деяких людей.

```
import pandas as pd

df = pd.DataFrame([(1, 'Ivanov', 'Ivan'),
                   (2, 'Petrov', 'Petro'),
```

```
                 (3, 'Kosovan', 'Vasyl'),
                 (4, 'Cherevko', 'Igor'),
                 (5, 'Kravtsov', 'Kyryl'),
                 (6, 'Rarenko', 'Anastasiya'),],
                columns=["id", "last_name", "first_name"])
df
```

Результат:

```
   id   last_name   first_name
0  1   Ivanov   Ivan
1  2   Petrov   Petro
2  3   Kosovan  Vasyl
3  4   Cherevko Igor
4  5   Kravtsov Kyryl
5  6   Rarenko  Anastasiya
```

Ви, напевно, пам'ятаєте, що таке первинний ключ в базі даних. Якщо ні, можна коротко згадати, що це одне (або декілька) полів (або стовпців) у таблиці бази даних, які унікально ідентифікують кожен її рядок.

«Індекс» для Pandas насправді означає щось більше, ніж «первинний ключ» у таблиці бази даних (хоча б з тим винятком, що можливі дублікати записів).

Тобто індекс (якщо він зроблений правильно, без дублікатів) є ідентифікатором для кожного рядка в базі даних. Ми можемо встановити індекс для одного з існуючих стовпців за допомогою виклику `.set_index()`.

Але тут потрібно бути дуже обережним з одним моментом. За замовчуванням більшість операцій Pandas, таких як `.set_index()` та багато інших, не виконуються на місці. Тобто, хоча виклик `df.set_index(«id»)` вище повертає копію фрейму даних df з індексом, встановленим для стовпця id (пам'ятайте, що Jupyter notebook відображає значення, яке повертається в останньому рядку комірки), оригінальний об'єкт `df` насправді тут не змінюється.

Якщо ми хочемо фактично змінити сам об'єкт `df`, потрібно використовувати прапор `inplace=True` для цих функцій (або присвоїти оригінальний об'єкт результату функції, але це не так чітко).

```
df.set_index("id", inplace=True)
df
```

Результат:

```
   last_name   first_name
id
1  Ivanov   Ivan
2  Petrov   Petro
3  Kosovan  Vasyl
4  Cherevko Igor
5  Kravtsov Kyryl
6  Rarenko  Anastasiya
```

Подивимось на іще деякі методи для роботи із фреймами в Pandas.

Доступ до окремих елементів можна отримати за допомогою нотації `.loc[row, column]`, де row позначає індекс, який ви шукаєте, а column - назву стовпця.

```
df.loc[1, "last_name"]
```

Результат:

```
'Ivanov'
```

Якщо ми хочемо отримати доступ до всіх прізвищ (або до всіх елементів у певному рядку), ми використовуємо символ-замінник :.

```
df.loc[:, "last_name"]
```

Результат:

```
id
1      Ivanov
2      Petrov
3     Kosovan
4    Cherevko
5    Kravtsov
6     Rarenko
Name: last_name, dtype: str
```

Ми можемо передати список потрібних стовпців, щоб отримати об'єкт типу DataFrame.

```
df.loc[:, ["last_name"]]
```

Результат:

```
    last_name
id
1  Ivanov
2  Petrov
3  Kosovan
4  Cherevko
5  Kravtsov
6  Rarenko
```

Те саме можна зробити з індексами рядків.

```
df.loc[[1,2],:]
```

Результат:

```
   last_name   first_name
id
1  Ivanov   Ivan
2  Petrov   Petro
```

Крім того, ми можемо використовувати '.loc' для зміни вмісту існуючих записів.

```
df.loc[6,"last_name"] = "Vikovan"
df
```

Результат:

```
   last_name   first_name
id
1  Ivanov   Ivan
2  Petrov   Petro
3  Kosovan  Vasyl
4  Cherevko Igor
5  Kravtsov Kyryl
6  Vikovan  Anastasiya
```

Ми можемо навіть додати додаткові рядки/стовпці.

```
df.loc[7,:] = ('Vikovan', 'Valentyn')
df
```

Результат:

```
last_name   first_name
id
1  Ivanov   Ivan
2  Petrov   Petro
3  Kosovan  Vasyl
4  Cherevko Igor
5  Kravtsov Kyryl
6  Vikovan  Anastasiya
7  Vikovan  Valentyn
```

☞ Наостанок пам'ятайте, що '.loc' завжди індексує на основі «індексу» (тобто, фактично, первинного ключа) фрейму даних разом з назвою стовпця. Якщо ви хочете отримати доступ на основі позиційного індексу (тобто, використовуючи лічильники з нульовим індексом як для рядків, так і для стовпців), ви можете скористатися властивістю `.iloc`.

```
df.iloc[4,1]
```

Результат:

```
'Kyryl'
```

2.2 Методи очищення даних та обробка відсутніх значень.

У реальному світі необроблені дані рідко бувають ідеальними. Вони часто містять помилки, невідповідності та відсутні значення.

Навіть найкращий алгоритм чи штучний інтелект не зможе побудувати хорошу модель, якщо навчальні дані містять помилки або пропуски. Наприклад, прогноз погоди буде ускладненим, якщо серед показників термометра за попередні дні затесалися такі дані як -500 градусів за Цельсієм, чи просто багато пропусків.

Очищення даних — це процес виявлення та виправлення цих проблем для поліпшення якості даних.

- *Дублікати*: це повторювані записи в наборі даних. Дублікати рядків можуть спотворити ваш аналіз або модель, особливо якщо вони представляють одну й ту саму подію або об'єкт.
- *Неправильні типи даних*: дані, які введені неправильно (наприклад, числа, збережені як рядки), можуть призвести до помилок в аналізі та навчанні моделі.
- *Випадкові значення*: екстремальні значення, які значно відрізняються від решти даних. Вони можуть спотворити результати статистичного аналізу або моделей машинного навчання.

Відсутність даних може відбуватися з різних причин, таких як помилки введення даних, несправності обладнання або проблеми з конфіденційністю. Важливо обережно поводитися з відсутніми значеннями, оскільки неправильне поводження може призвести до упереджених результатів.

Типи відсутніх даних:

- *MCAR* (Missing Completely at Random, повністю випадкова відсутність): ймовірність відсутності даних не залежить від самих даних.
- *MAR* (відсутність випадкова): ймовірність відсутності даних пов'язана з іншими спостережуваними даними, але не з самими відсутніми даними.
- *MNAR* (відсутність невипадкова): відсутність пов'язана з самими неспостережуваними даними.

2.2.1 Видалення відсутніх даних.

Видалення рядків або стовпців з відсутніми значеннями є найпростішим підходом, але може призвести до втрати цінної інформації.

У наступному прикладі метод `df.dropna()` видаляє всі рядки, у яких є хоча б одне відсутнє значення, а метод `df.dropna(subset=['feature1', 'feature2'])` видаляє рядки, у яких є хоча б одне відсутнє значення в числових стовпцях feature1 та feature2.

```
import pandas as pd

df = pd.read_csv('../resources/data.csv')
print(df, '\n')

df_cleaned = df.dropna()
df_feature_cleaned = df.dropna(subset=['feature1', 'feature2'])
print(df_cleaned, '\n')
print(df_feature_cleaned, '\n')
```

Результат:

```
   feature1  feature2 categorical_column  target
0       1.2       3.2                 A      10
1       2.3       NaN                 B      15
2       3.1       1.1                 C       7
3       4.7       4.8                 A      18
4       NaN       2.7                 B      11
5       5.1       4.2                 C      19
6       2.8       NaN                 A      13
7       4.1       3.9                 B      16
8       5.3       4.1               NaN      14
9       3.3       2.1                 C      12

   feature1  feature2 categorical_column  target
0       1.2       3.2                 A      10
2       3.1       1.1                 C       7
3       4.7       4.8                 A      18
5       5.1       4.2                 C      19
7       4.1       3.9                 B      16
9       3.3       2.1                 C      12

   feature1  feature2 categorical_column  target
0       1.2       3.2                 A      10
2       3.1       1.1                 C       7
3       4.7       4.8                 A      18
5       5.1       4.2                 C      19
7       4.1       3.9                 B      16
8       5.3       4.1               NaN      14
9       3.3       2.1                 C      12
```

2.2.2 Заповнення середнім.

При заповненні середнім (Mean Imputation) ми замінюємо відсутні значення середнім значенням стовпця.

```
df['feature1'].fillna(df['feature1'].mean(), inplace=True)
df['feature2'].fillna(df['feature2'].mean(), inplace=True)
df
```

Результат:

```
feature1 feature2 categorical_column   target
0  1.200000 3.2000   A  10
1  2.300000 3.2625   B  15
2  3.100000 1.1000   C  7
3  4.700000 4.8000   A  18
4  3.544444 2.7000   B  11
5  5.100000 4.2000   C  19
6  2.800000 3.2625   A  13
7  4.100000 3.9000   B  16
8  5.300000 4.1000   NaN   14
9  3.300000 2.1000   C  12
```

2.2.3 Заміщення медіанним значенням.

При заміщенні медіанним значенням (Median Imputation) ми заміщуємо відсутні значення медіанним значенням. Цей метод є корисним, коли дані є викривленими.

```
df = pd.read_csv('../resources/data.csv')
df['feature1'].fillna(df['feature1'].median(), inplace=True)
df
```

Результат:

```
   feature1 feature2 categorical_column   target
0  1.2   3.2   A  10
1  2.3   NaN   B  15
2  3.1   1.1   C  7
3  4.7   4.8   A  18
4  3.3   2.7   B  11
5  5.1   4.2   C  19
6  2.8   NaN   A  13
7  4.1   3.9   B  16
8  5.3   4.1   NaN   14
9  3.3   2.1   C  12
```

2.2.4 Заміщення модою.

Заміщення модою (Mode Imputation) часто використовується для категоріальних даних.

```
df['categorical_column'].fillna(df['categorical_column'].mode()[0], inplace=True)
df
```

Результат:

```
feature1 feature2 categorical_column   target
```

```
0  1.2   3.2   A  10
1  2.3   NaN   B  15
2  3.1   1.1   C  7
3  4.7   4.8   A  18
4  3.3   2.7   B  11
5  5.1   4.2   C  19
6  2.8   NaN   A  13
7  4.1   3.9   B  16
8  5.3   4.1   A  14
9  3.3   2.1   C  12
```

☞ Інші поширені методи заповнення даних:

Імпутування K-найближчих сусідів (KNN): оцінює відсутні значення на основі значень найближчих сусідів.

Багатовимірне імпутування (підстановка): враховує взаємозв'язки між ознаками для імпутування відсутніх значень.

Створення індикаторних змінних: інший підхід полягає у створенні бінарної індикаторної змінної, яка позначає наявність відсутніх даних.

2.3 Масштабування та нормалізація даних.

Масштабування та нормалізація даних (Data Scaling and Normalization) є важливими етапами попередньої обробки, які можуть суттєво вплинути на ефективність ваших моделей.

Масштабування даних гарантує, що всі ознаки однаково впливають на процес навчання моделі. Коли ознаки мають дуже різні діапазони, ті, що мають більші діапазони, можуть домінувати в обчисленнях відстані або кроках градієнта, що призводить до неоптимальних моделей.

Алгоритми на основі відстані: В алгоритмах, таких як K-найближчі сусіди (KNN) або машини опорних векторів (SVM), відстань між точками даних має вирішальне значення. Якщо одна ознака має набагато більший діапазон, ніж інші, вона непропорційно впливатиме на метрику відстані, що може спотворити результати.

Градієнтний спуск: Алгоритми, такі як лінійна регресія та нейронні мережі, використовують градієнтний спуск для мінімізації функції вартості. Якщо ознаки не масштабуються, алгоритм градієнтного спуску може потребувати більше часу для збіжності або може збігатися до неоптимального рішення через нерівномірні кроки, зроблені в просторі параметрів.

2.3.1 Мінімально-максимальне масштабування.

Мінімально-максимальне масштабування (min-max normalization) перемасштабовує дані до фіксованого діапазону, зазвичай `[0, 1]`. Ця техніка особливо корисна, коли ви знаєте, що ваші дані відповідають розподілу з чіткими верхньою та нижньою межами.

$$X' = \frac{X - X_{min}}{X_{max} - X_{min}}$$

- Використовуйте мінімально-максимальне масштабування, коли ви хочете, щоб усі ваші ознаки мали однаковий масштаб (наприклад, в алгоритмах, таких як KNN, SVM).
- Особливо корисно, коли дані розподілені по відомому і фіксованому діапазону.

```
from sklearn.preprocessing import MinMaxScaler
import pandas as pd
```

```
data = {'feature': [10, 50, 100, 150, 200]}
df = pd.DataFrame(data)

scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(df)

print(scaled_data)
```

Результат:

```
[[0.       ]
 [0.21052632]
 [0.47368421]
 [0.73684211]
 [1.       ]]
```

Ми можемо вибрати окремі стовпці для масштабування через `numerical_scaled = scaler.fit_transfo` В наступному прикладі ми спочатку вибираємо числові стовпці для масштабування через `numerical_cols =` потім розділяємо стовпці категорій та цільові стовпці `categorical_cols = df[['categorical_column', 't` ініціалізуємо масштабування `scaler = MinMaxScaler()`, підготовуємо та перетворюємо числові стовпці `numerical_scaled = scaler.fit_transform(numerical_cols)`, перетворюємо нормалізовані дані назад у DataFrame `numerical_scaled_df = pd.DataFrame(numerical_scaled, columns=numeri` та об'єднуємо нормалізовані числові дані з категоріальними стовпцями `df_scaled = pd.concat([numerica`

```
df = pd.read_csv('../resources/data.csv')

numerical_cols = df[['feature1', 'feature2']]

categorical_cols = df[['categorical_column', 'target']]

from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

numerical_scaled = scaler.fit_transform(numerical_cols)

numerical_scaled_df = pd.DataFrame(numerical_scaled,
    columns=numerical_cols.columns)

df_scaled = pd.concat([numerical_scaled_df,
    categorical_cols.reset_index(drop=True)], axis=1)

print(df_scaled)
```

Результат:

	feature1	feature2	categorical_column	target
0	0.000000	0.567568	A	10
1	0.268293	NaN	B	15
2	0.463415	0.000000	C	7
3	0.853659	1.000000	A	18
4	NaN	0.432432	B	11
5	0.951220	0.837838	C	19

```
6  0.390244       NaN                A      13
7  0.707317  0.756757                B      16
8  1.000000  0.810811              NaN      14
9  0.512195  0.270270                C      12
```

2.3.2 Нормалізація (стандартизація) за Z-балом.

Нормалізація (стандартизація) за Z-балом (Z-Score Normalization or standardization) перемасштабовує дані так, щоб середнє значення дорівнювало 0, а стандартне відхилення — 1. Ця техніка корисна, коли дані мають гаусівський (нормальний) розподіл, але не обов'язково обмежені.

$$X' = \frac{X - \nu}{\sigma}$$

- Використовуйте нормалізацію за Z-балом, коли ваші дані мають нормальний розподіл.
- Це стандартний вибір для багатьох моделей машинного навчання, особливо тих, що передбачають нормальний розподіл даних.

```python
from sklearn.preprocessing import StandardScaler
import pandas as pd

data = {'feature': [10, 50, 100, 150, 200]}
df = pd.DataFrame(data)

scaler = StandardScaler()
standardized_data = scaler.fit_transform(df)

print(standardized_data)
```

Результат:

```
[[-1.35411306]
 [-0.76536825]
 [-0.02943724]
 [ 0.70649377]
 [ 1.44242478]]
```

2.3.3 Надійне масштабування.

Надійне масштабування (Robust Scaling) використовує медіану та міжквартильний розмах (IQR) для масштабування, що робить його надійним щодо винятків. На відміну від мінімально-максимального масштабування та нормалізації Z-балу, надійне масштабування менше піддається впливу екстремальних значень у даних.

$$X' = \frac{X - \text{медіана}(X)}{\text{IQR}(X)}$$

Де $\text{IQR}(X)$ — діапазон між 25-м процентилем ($Q1$) і 75-м процентилем ($Q3$).

Використовуйте надійне масштабування, якщо ваші дані містять винятки або мають негаусівський розподіл. Воно ідеально підходить для ситуацій, коли дані є асиметричними або містять екстремальні значення, які не повинні сильно впливати на масштабування.

```
from sklearn.preprocessing import RobustScaler
import pandas as pd

data = {'feature': [10, 50, 90, 100, 110, 150, 160, 200]}
df = pd.DataFrame(data)

scaler = RobustScaler()
robust_scaled_data = scaler.fit_transform(df)

print(robust_scaled_data)
```

Результат:

```
[[-1.42857143]
 [-0.85714286]
 [-0.28571429]
 [-0.14285714]
 [ 0.        ]
 [ 0.57142857]
 [ 0.71428571]
 [ 1.28571429]
 [34.14285714]]
```

☞ Для яких випадків підходить той чи інший метод масштабування?

Мінімально-максимальне масштабування: використовується, коли відомі межі даних або коли алгоритм вимагає даних у певному діапазоні.

Нормалізація за Z-балом: підходить для нормально розподілених даних або коли потрібно стандартизувати ознаки, щоб вони мали однакову важливість.

Надійне (Robust) масштабування: найкраще підходить для роботи з даними, що м істять винятки.

2.4 Вступ до конструювання ознак.

Інженерія ознак (Feature Engineering) є важливим етапом у процесі попередньої обробки даних. Вона передбачає перетворення необроблених даних на ознаки, які краще відображають основну проблему для моделей прогнозування. Якісна інженерія ознак може значно покращити ефективність моделей машинного навчання, надаючи їм найбільш релевантні та інформативні вхідні дані.

Для прикладу, маючи такі фізичні дані про людину як її вага та зріст, ми можемо сконструювати нову ознаку - "індекс маси тіла що буде значно користнішим для нашої моделі, ніж дві інші ознаки кожна окремо.

Інженерія ознак - це процес відбору, модифікації або створення нових ознак з необроблених даних для покращення ефективності моделей машинного навчання. Метою є створення ознак, які ефективніше відображають основні закономірності в даних, ніж вихідні необроблені дані. Правильна інженерія ознак може призвести до спрощення моделей, підвищення точності та скорочення часу навчання.

2.4.1 Створення нових ознак.

Створення нових ознак це виведення нових змінних з існуючих, які краще відображають основну структуру даних. Це може включати математичні перетворення, агрегації або операції,

специфічні для певної області.

В наступному прикладі, маючи такі дані як вага та ріст людини, ми створюємо нову ознаку: BMI (body mass index, індекс маси тіла).

```
import pandas as pd

data = {
    'height': [1.60, 1.75, 1.82, 1.90],
    'weight': [55, 80, 72, 90]
}
df = pd.DataFrame(data)

df

df['BMI'] = df['weight'] / (df['height'] ** 2)

df
```

Результат:

```
   height   weight   BMI
0  1.60  55 21.484375
1  1.75  80 26.122449
2  1.82  72 21.736505
3  1.90  90 24.930748
```

2.4.2 Кодування категоріальних змінних.

Алгоритми машинного навчання зазвичай вимагають числових даних. Тому категоріальні змінні необхідно перетворити в числовий формат. Існує кілька методів кодування категоріальних змінних:

Кодування *One-hot* (унітарне кодування) перетворює кожну категорію в новий бінарний стовпець. Цей метод підходить для випадків, коли кількість категорій обмежена.

```
data = {'color': ['red', 'blue', 'green']}
df = pd.DataFrame(data)

# One-Hot Encoding
df_encoded = pd.get_dummies(df, columns=['color'])

print(df_encoded)
```

Результат:

```
   color_blue  color_green  color_red
0           0            0          1
1           1            0          0
2           0            1          0
```

Кодування міток (Label Encoding) присвоює кожної категорії унікальне ціле число. Це простий метод, але він передбачає порядкові відносини між категоріями, що може бути недоречним у деяких випадках.

```
from sklearn.preprocessing import LabelEncoder

data = {'color': ['red', 'blue', 'green']}
df = pd.DataFrame(data)

# Label Encoding
encoder = LabelEncoder()
df['color_encoded'] = encoder.fit_transform(df['color'])

df
```

Результат:

```
   color color_encoded
0  red   2
1  blue  0
2  green 1
```

Цільове кодування (Target Encoding) передбачає заміну кожної категорії середнім значенням цільової змінної для цієї категорії. Ця техніка є корисною, коли категоріальна змінна має велику кількість рівнів.

```
data = {
    'color': ['red', 'blue', 'green', 'red', 'blue', 'green'],
    'target': [1, 0, 0, 1, 1, 0]
}
df = pd.DataFrame(data)

# Target Encoding
df['color_encoded'] = df.groupby('color')['target'].transform('mean')

print(df)
```

Результат:

```
   color  target  color_encoded
0    red       1            1.0
1   blue       0            0.5
2  green       0            0.0
3    red       1            1.0
4   blue       1            0.5
5  green       0            0.0
```

2.4.3 Функція взаємодії.

Функція взаємодії створюються шляхом поєднання двох або більше змінних для відображення взаємодії між ними. Ці функції можуть виявити взаємозв'язки, які не є очевидними при розгляді змінних окремо.

Приклад:

Припустимо, що ви маєте такі дані, як вік і дохід користувачів. Термін взаємодії можна створити шляхом множення цих двох властивостей, щоб відобразити, як дохід змінюється з віком.

В наступному прикладі ми створюємо функцію взаємодії, яка включає в себе параметри вік та дохід.

```
data = {
    'age': [25, 35, 45, 55],
    'income': [30000, 50000, 70000, 90000]
}
df = pd.DataFrame(data)

df['age_income_interaction'] = df['age'] * df['income']

df
```

Результат:

```
   age   income   age_income_interaction
0  25 30000 750000
1  35 50000 1750000
2  45 70000 3150000
3  55 90000 4950000
```

2.4.4 Поліноміальні ознаки.

Створюються шляхом генерації нових ознак, які є поліноміальними комбінаціями існуючих ознак. Це може бути особливо корисно при моделюванні нелінійних взаємозв'язків.

Приклад:

Припустимо, у вас є одна ознака x. Ви можете створити поліноміальні ознаки, такі як x^2, x^3 тощо, які можуть допомогти у виявленні нелінійних взаємозв'язків у даних.

```
from sklearn.preprocessing import PolynomialFeatures

data = {'x': [2, 3, 4]}
df = pd.DataFrame(data)

poly = PolynomialFeatures(degree=3, include_bias=False)
poly_features = poly.fit_transform(df)

df_poly = pd.DataFrame(poly_features, columns=poly.get_feature_names_out(['x']))

df_poly
```

Результат:

```
   x  x^2   x^3
0  2.0   4.0   8.0
1  3.0   9.0   27.0
2  4.0   16.0  64.0
```

☞ Після завершення обробки необхідно переконатися, що дані не містять пропущених значень, дублікатів, некоректних типів даних та аномальних викидів, а всі ознаки мають очікуваний формат і масштаб. Для цього використовують статистичний аналіз, перевірку структури даних, візуалізацію та засоби бібліотеки pandas, такі як `info()`, `describe()`, `isnull()` і `duplicated()`.

Якісно підготовлені дані дозволяють моделям машинного навчання точніше виявляти закономірності, швидше навчатися, бути менш чутливими до шуму та краще узагальнювати знання на нових даних, що безпосередньо підвищує точність, стійкість і надійність прогнозування.

✎ **Питання для самоконтролю.**

1. Що таке обробка даних (Data Processing) та яке її значення в процесі інтелектуального аналізу даних?
2. Які основні етапи включає процес обробки даних?
3. Чому якість обробки даних впливає на результати машинного навчання?
4. Які основні проблеми можуть бути присутні у «сирих» даних?
5. Що таке бібліотека pandas та для чого вона використовується?
6. Які основні структури даних підтримує бібліотека pandas?
7. Що таке DataFrame?
8. Чим DataFrame відрізняється від Series?
9. Як створити DataFrame у Python?
10. Як зчитати дані у DataFrame з CSV-файлу?
11. Як переглянути перші та останні записи DataFrame?
12. Як отримати інформацію про структуру DataFrame?
13. Як отримати статистичний опис числових даних за допомогою pandas?
14. Як вибрати окремий стовпець або кілька стовпців DataFrame?
15. Яка різниця між методами ‘loc[]‘ та ‘iloc[]‘?
16. Як відфільтрувати рядки DataFrame за певною умовою?
17. Як додати новий стовпець до DataFrame?
18. Як видалити стовпець або рядок із DataFrame?
19. Як перейменувати стовпці DataFrame?
20. Як відсортувати дані за одним або кількома стовпцями?
21. Що таке очищення даних (Data Cleaning)?
22. Які основні причини виникнення неякісних даних?
23. Що таке пропущені значення (Missing Values)?
24. Які причини появи пропущених значень у наборах даних?
25. Як виявити пропущені значення у DataFrame?
26. Які методи використовуються для видалення пропущених значень?
27. Які методи використовуються для заповнення пропущених значень?
28. У яких випадках доцільно видаляти рядки з пропущеними значеннями?

29. Що таке імпутація даних (Data Imputation)?
30. Які статистичні методи використовуються для заповнення пропущених значень?
31. Що таке дублікати даних?
32. Як знайти дублікати у DataFrame?
33. Як видалити дублікати за допомогою pandas?
34. Що таке викиди (Outliers)?
35. Якими методами можна виявляти викиди?
36. Чому викиди можуть негативно впливати на результати аналізу даних?
37. Що таке масштабування даних (Data Scaling)?
38. Для чого виконується масштабування ознак перед навчанням моделей?
39. У чому полягає різниця між масштабуванням та нормалізацією даних?
40. Що таке мінімально-максимальна нормалізація?
41. Як обчислюється мінімально-максимальна нормалізація?
42. Що таке стандартизація (Standardization)?
43. Як обчислюється стандартизація за Z-оцінкою (Z-score)?
44. Для яких алгоритмів машинного навчання масштабування є особливо важливим?
45. Які переваги має нормалізація даних?
46. Які недоліки має мінімально-максимальна нормалізація?
47. Які переваги має стандартизація порівняно з мінімально-максимальною нормалізацією?
48. Які інструменти бібліотеки scikit-learn використовуються для масштабування та нормалізації даних?
49. Як перевірити, що дані були успішно очищені та підготовлені до аналізу?
50. Яким чином правильна обробка даних впливає на точність, стійкість і узагальнювальну здатність моделей машинного навчання?

Розділ 3

Візуалізація даних.

Людський мозок значно краще сприймає візуальні образи, ніж набори сухих чисел. За лічені секунди можна помітити тенденцію на графіку, яку не можна було б побачити в таблиці в екселі чи в базі даних. Саме тому надзвичайно важливим етапом аналізу даних є візуалізація.

Важливо підкреслити різницю між двома дуже різними поняттями, які люди описують, використовуючи загальний термін "візуалізація". Перше з них — "візуалізація для дослідження даних": використання візуалізації для розуміння (тобто для власного розуміння) сукупності даних з метою подальшого аналізу. Коротко кажучи, мета цього типу візуалізації — "знайти істину"в даних. Альтернативою є "візуалізація для презентації": виокремлення певного аспекту даних у вигляді зрозумілої діаграми або графіку, що передає широкій аудиторії конкретний аспект даних, який ви хочете донести. Цей тип візуалізації має на меті "переконати інших людей у правдивості ваших висновків".

Візуалізація VS статистика: Візуалізація майже завжди надає більш інформативний (хоча і менш кількісний) огляд ваших даних, ніж статистика (підхід, а не галузь науки в цілому).

3.1 Типи даних.

Перед тим, як приступити до візуалізації, варто зрозуміти, з яким типом даних ви маєте справу. Це потрібно для того, щоб, у першу чергу, не помилитися із вибором методу візуалізації.

Чотири основні типи даних:

- *Номінальні дані (Nominal data)*: категоріальні дані без внутрішнього порядку між категоріями. Наприклад, змінна "тип домашньої тварини"може складатися з класів собака, кіт, кролик, і між цими двома типами немає відносного порядку, це просто різні дискретні значення.
- *Порядкові дані (Ordinal data)*: категоріальні дані з внутрішньою послідовністю, але "відмінності"між категоріями не мають суто числового значення. Канонічним прикладом тут є відповіді на опитування з такими відповідями: категорично не згоден, трохи не згоден, нейтральний, трохи згоден, категорично згоден. Важливою особливістю тут є те, що, хоча між цими типами існує чітка послідовність, немає сенсу вважати, що різниця між "трохи згоден"і "категорично згоден"є "такою ж як різниця між "нейтральний"і "трохи згоден".
- *Інтервальні дані (Interval data)*: числові дані, тобто дані, які можна відобразити на "числовій прямій"; важливим аспектом на відміну від порядкових даних є не "дискретна проти безперервної диференціації"(наприклад, цілочисельні значення можна вважати інтервальними даними), а той факт, що відносні відмінності в інтервальних даних мають значення. Класичним прикладом є температура (у Фаренгейті або Цельсії, що ми підкреслимо трохи пізніше): тут різниці між температурами мають значення: 10 і 15 градусів розділені такою

ж величиною, як 15 і 20 (ця властивість настільки властива числовим даним, що майже дивно її підкреслювати). З іншого боку, інтервальні дані охоплюють випадки, коли нульова точка "не має реального значення"; на практиці це означає, що співвідношення між двома точками даних не має значення. Двадцять градусів за Фаренгейтом не є "вдвічі гарячішими"в будь-якому значущому сенсі, ніж 10 градусів; і, звичайно, не нескінченно гарячішими, ніж нуль градусів.

- *Дані про співвідношення (Ratio data)*: також числові дані, але співвідношення між вимірами має певне значення. Класичним прикладом тут є температура за Кельвіном. Очевидно, що, як і температура за Фаренгейтом або Цельсієм, це описує основне явище температури, але на відміну від попередніх випадків, нуль за Кельвіном має значення з точки зору молекулярної енергії в речовині (тобто її відсутність). Це означає, що співвідношення мають реальне значення: речовина при 20 градусах Кельвіна має вдвічі більше кінетичної енергії на молекулярному рівні, ніж та сама речовина при 10 градусах Кельвіна.

✎ "Допустимі операції які ми можемо виконувати над двома елементами різних типів:

Номінальні дані: $=,\neq$. Все, що ми можемо зробити з номінальними даними, це порівняти два дані та перевірити, чи вони рівні.

Порядкові дані: $=,\neq,<,>$. Окрім перевірки на рівність, ми також можемо порівнювати порядок різних точок даних.

Інтервальні дані: $=,\neq,<,>,-$. Ми маємо всі операції з порядковими даними, але також можемо обчислювати точні числові різниці між двома точками даних. Залежно від контексту, ми можемо сказати, що додавання також дозволено, але іноді додавання фактично передбачає нульову точку і тому застосовується тільки до даних про співвідношення.

Дані співвідношення: $=,\neq,<,-,+,\div$. Ми маємо всі операції з інтервальними даними, але додавання тепер практично завжди дозволено, і ми можемо додатково виконувати ділення (для визначення співвідношень) між різними точками даних.

Щоб бути більш формальними, далі ми припускаємо, що маємо доступ до набору даних, який можна позначити як

$\{x^{(1)}, x^{(2)}, \ldots, x^{(m)}\}$

де кожне $x^{(i)}$ є деякою точкою даних. $x^{(i)} = (a_1^i, a_2^i, a_3^i, ...)$

Для побудови всіх цих графіків ми будемо використовувати бібліотеку matplotlib, яка добре інтегрується з Jupyter notebook.

3.2 1D дані

3.2.1 Стовпчасті діаграми — категоріальні дані

Якщо ваші дані є одновимірними категоріальними, єдиним реальним варіантом для їх візуалізації є використання стовпчастої діаграми (Bar charts). Зверніть увагу, що, виходячи з нашого припущення про те, що порядок точок даних у нашому наборі даних не має значення, єдиною значущою інформацією, якщо ми маємо набір даних типу

$\{student, teacher, student, professor, teacher, student, \ldots\}$

буде кількість повторень кожного елемента. Таким чином, ми можемо ефективно узагальнити дані, включивши кількість повторень кожного значення даних (це легко зробити за допомогою класу ‘collections.Counter‘).

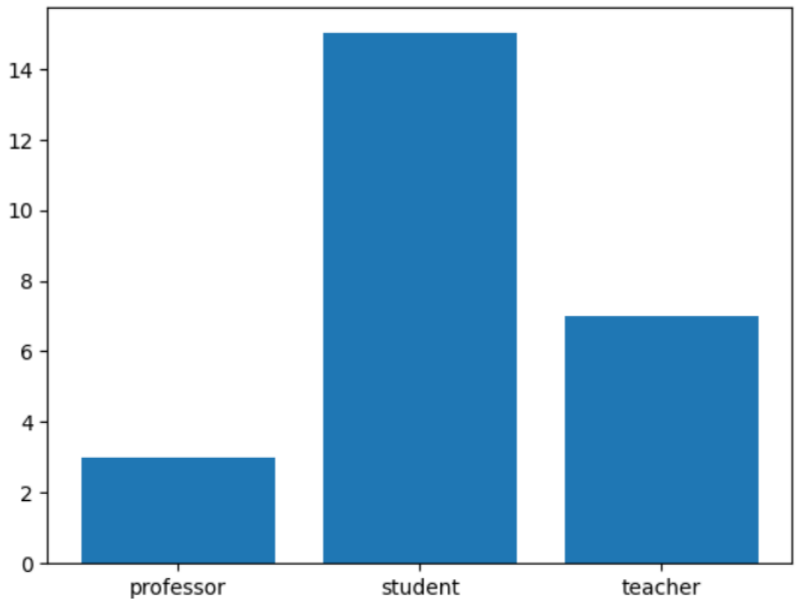
Примітка: незважаючи на те, що ви, можливо, бачили на деяких рисунках, зверніть увагу, що включення ліній між різними стовпчиками однозначно не має сенсу.

```
import collections
import matplotlib.pyplot as plt
```

```
import numpy as np

data = np.random.permutation(np.array(["student"]*15 + ["teacher"]*7 +
    ["professor"]*3))
# print(data)
counts = collections.Counter(data)
plt.bar(range(len(counts)), list(counts.values()), tick_label=list(counts.keys()))
```

Результат:



3.2.2 Кругові діаграми.

Кругові діаграми (Pie charts) призначені для представлення даних, а не для їх дослідження!

Люди не дуже добре оцінюють площу або кути (наприклад, важко визначити, чи один сегмент становить 23 %, а інший — 27 %). Стовпчасті діаграми набагато простіші, оскільки люди точніше порівнюють довжини.

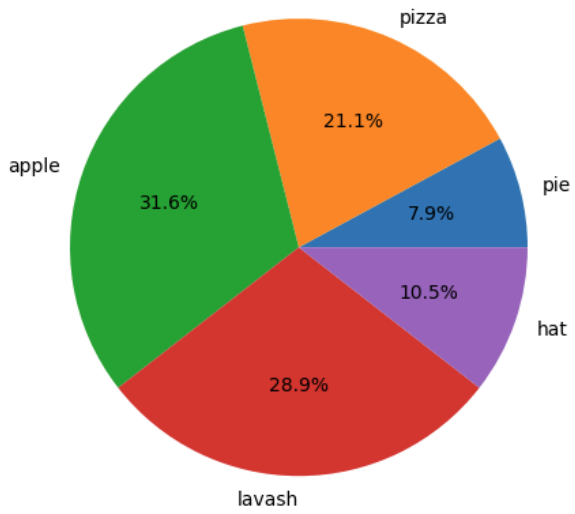
Порівняти дві різні кругові діаграми (наприклад, два періоди часу) майже неможливо. Неможливо легко додати смуги похибки, опорні лінії або точні порівняння.

Якщо ж ви, з якоїсь причини, вирішили побудувати кругову діаграму (наприклад, щоб прпродемонструвати одемонструвтаи клієнту), ось приклад того, як це можна зробити:

```
data = {"pie": 3,
        "pizza": 8,
        "apple": 12,
        "lavash": 11,
        "hat": 4}

plt.pie(data.values(), labels=data.keys(), autopct='%1.1f%%')
plt.axis('equal')
```

Результат:



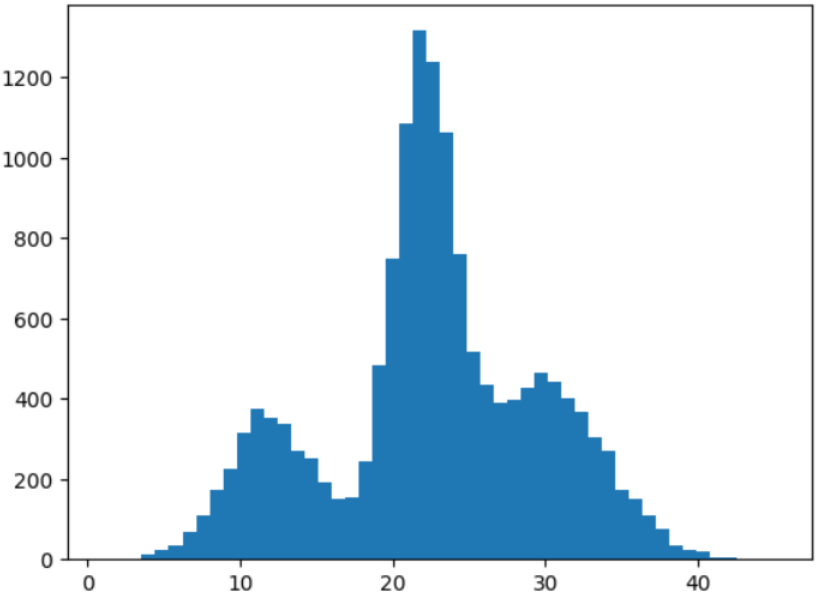
3.2.3 Гістограми — числові дані

Гістограма (Histogram) — це графік, який відображає розподіл числових даних, показуючи кількість спостережень у кожному інтервалі (біні).

```
np.random.seed(0)
data = np.concatenate([30 + 4*np.random.randn(5000),
                       22 + 2*np.random.randn(7000),
                       12 + 3*np.random.randn(3000)])
plt.hist(data, bins=50)


# data = np.concatenate([10 + 5*np.random.randn(10000)])
# plt.hist(data, bins=50);
```

Результат:



Гістограма показує розподіл числових даних, дозволяючи оцінити частоту появи значень у

різних інтервалах. Бін (Bin) — це інтервал значень, у межах якого підраховується кількість спостережень. Занадто мала кількість інтервалів приховує деталі розподілу, тоді як занадто велика може зробити гістограму шумною та ускладнити її аналіз.

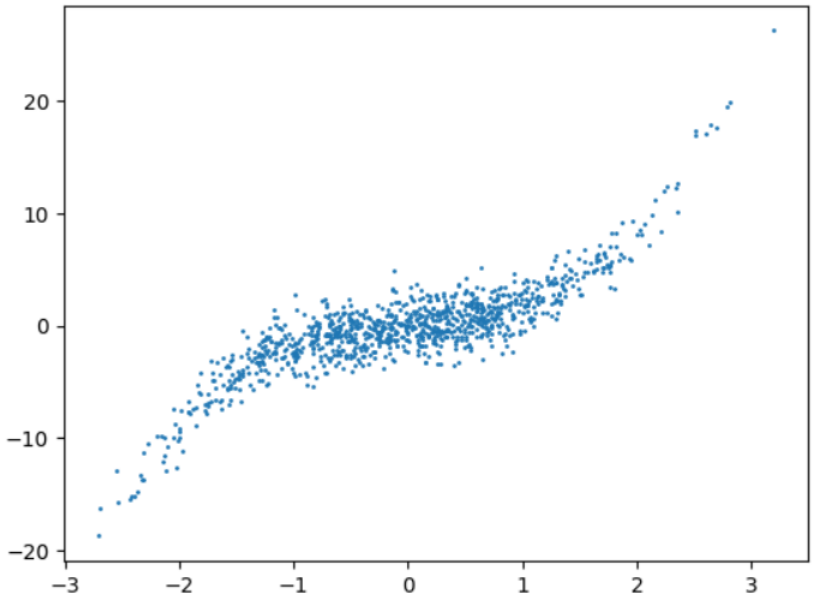
3.3 2D дані

3.3.1 Точкові діаграми — числові x числові.

Якщо обидва виміри даних є числовими, найприроднішим першим типом діаграми, який слід розглянути, є точкова діаграма (scatter plot): нанесення точок, які просто відповідають різним координатам даних. (Діаграма розсіювання)

```
x = np.random.randn(1000)
y = 0.8*x**3 + x + 1.5*np.random.randn(1000)
plt.scatter(x,y,s=1)
```

Результат:



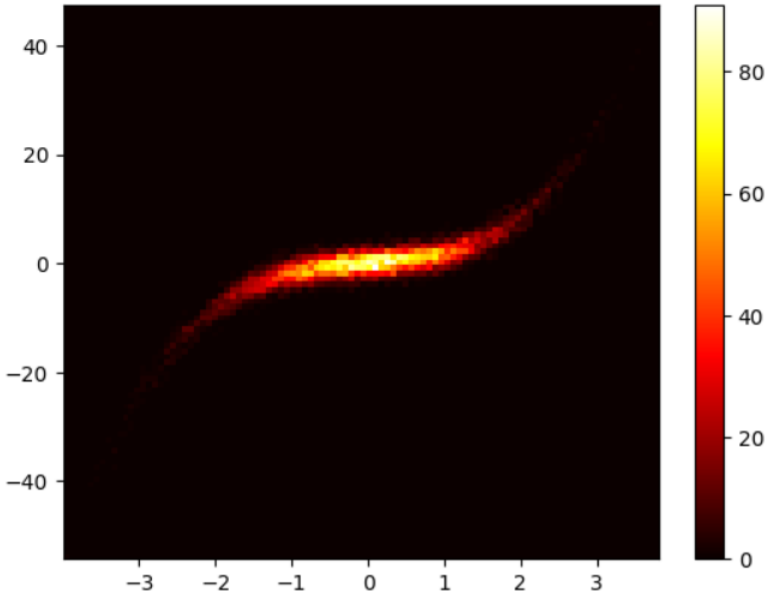
Однак існує також природний режим відмови, коли точок занадто багато, щоб їх можна було чітко розділити, і графік втрачає здатність показувати щільність даних. Наприклад, якщо у нас в 10 разів більше точок, графік вже не так чітко показує щільність у внутрішній частині "щільності".

У цьому випадку надлишку даних ми також можемо створити 2D гістограму даних (яка групує дані за обома вимірами) і вказати "висоту"кожного блоку за допомогою кольорової карти. Такі графіки можуть чіткіше вказувати щільність точок у регіонах, які в оригінальному графіку розсіювання мають суцільний колір. Ці 2D гістограми іноді також називають *тепловими картами* (heat maps), але ця назва часто конфліктує з подібними версіями, що використовуються для побудови 2D категоріальних даних, тому тут ми будемо використовувати термін *2D гістограма*.

```
x = np.random.randn(10000)
y = 0.8*x**3 + x + 1.5*np.random.randn(10000)
#plt.scatter(x,y,s=10)
```

```
plt.hist2d(x,y,bins=100);
plt.colorbar();
plt.set_cmap('hot')
```

Результат:



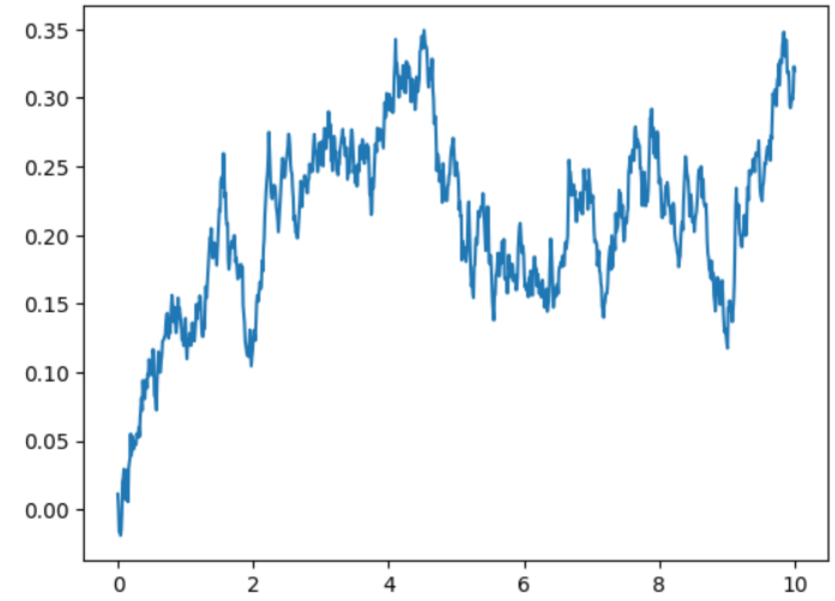
3.3.2 Лінійна діаграма - числові x числові.

Відомий всім зі школи і зрозумілий приклад візуалізації двовимірних числових даних, це лінійна діаграма (або графік, line plot). Це тип візуалізації, який відображає дані у вигляді точок, з'єднаних відрізками прямих.

```
x = np.linspace(0,10,1000)
y = np.cumsum(0.01*np.random.randn(1000))
plt.plot(x,y)
```

Результат:

```
[<matplotlib.lines.Line2D at 0x1957fbbcb50>]
```



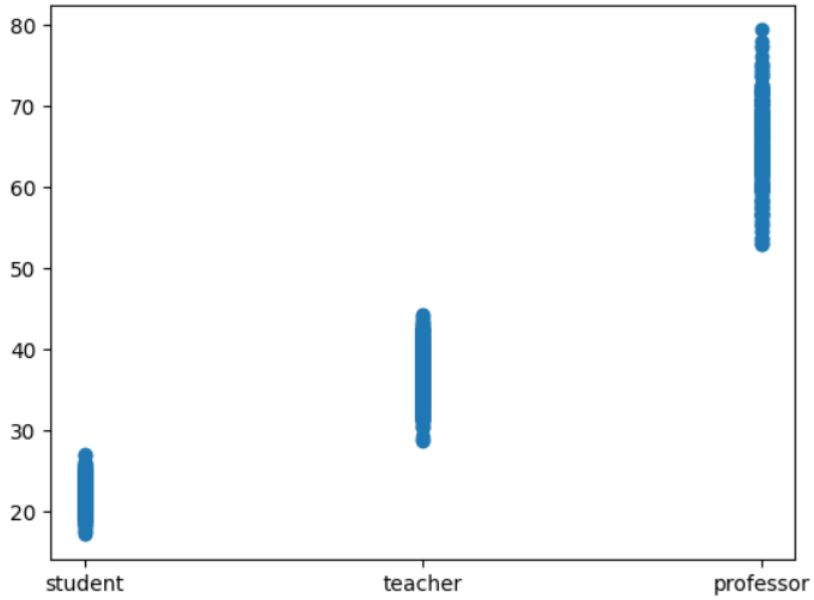
3.3.3 Коробкова і скрипкові діаграми — категоріальні x числові

Розглянемо простий приклад, де (тільки з вигаданими даними) ми будуємо графік учасників навчального процесу та їхній вік.

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.linalg as la

data= {"student": 22 + 1.4*np.random.randn(1000),
       "teacher": 37 + 2.5*np.random.randn(900),
       "professor": 65 + 5.5*np.random.randn(300)}
plt.scatter(np.concatenate([i*np.ones(len(x)) for i,x in
    enumerate(data.values())]),
            np.concatenate(list(data.values())))
plt.xticks(range(len(data)), data.keys());
```

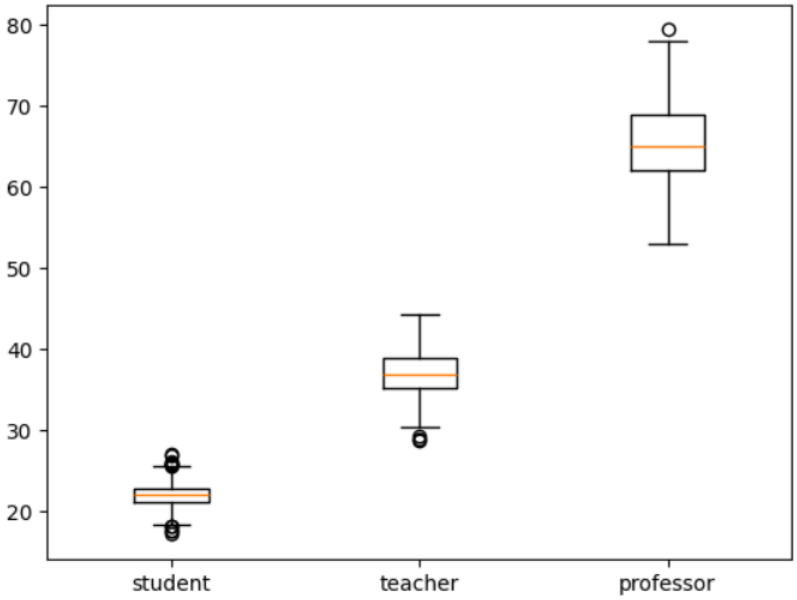
Результат:



Очевидно, що, дивлячись тільки на цей графік, можна отримати дуже мало, оскільки в щільній лінії точок недостатньо інформації, щоб дійсно зрозуміти розподіл числової змінної для кожної точки. Поширеною стратегією тут є використання графіка коробкова, або коробка з вусами (box and whiskers), яка відображає медіану даних (як лінію в середині коробки), 25-й і 75-й процентилі даних (як нижня і верхня межі коробки), "вуса"встановлюються за допомогою ряду різних можливих угод (за замовчуванням Matplotlib використовує 1,5-кратний інтерквартильний розмах, відстань між 25-м і 75-м процентилем), а будь-які точки поза цим діапазоном ("випадкові значення або "викиди") відображаються окремо.

```
plt.boxplot(data.values())
plt.xticks(range(1,len(data)+1), data.keys())
```

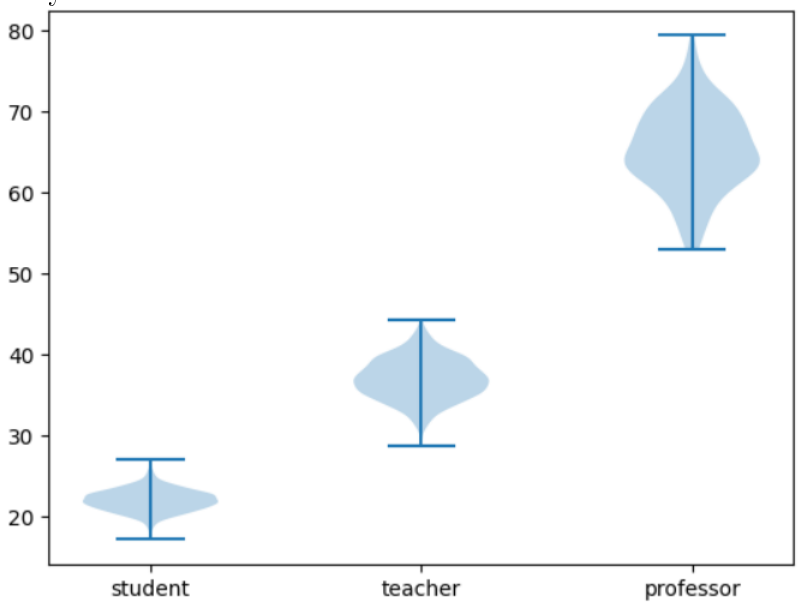
Результат:



☞ Звичайно, так само як середні значення та стандартні відхилення не описують повністю набір даних, статистика бокс-плат і вусів не відображає повністю розподіл даних. З цієї причини також часто використовуються скрипкові діаграми (violin plots), які створюють міні-гістограми.

```
plt.violinplot(data.values())
plt.xticks(range(1,len(data)+1), data.keys())
```

Результат:



3.3.4 Теплова карта та бульбашкові діаграми — категоріальні x категоріальні

Теплова карта (Heatmap) — це спосіб візуалізації даних, у якому значення відображаються за допомогою кольорів, що дозволяє швидко оцінити їхню величину та закономірності. За допомогою теплових карт найчастіше візуалізують матриці кореляцій, статистичні таблиці, матриці відстаней, частоти спостережень та інші двовимірні числові дані. Теплова карта кореляцій використовується для наочного відображення сили та напряму кореляції між числовими ознаками, що допомагає виявити сильно пов'язані або незалежні змінні.

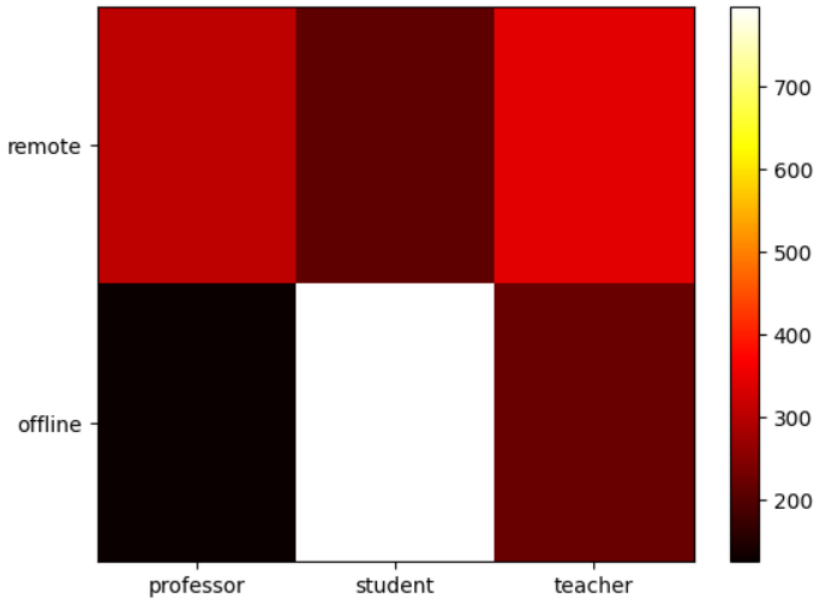
Коли обидва виміри наших 2D-даних є категоріальними, ми маємо ще менше інформації для використання. Знову ж таки, метою буде надання певної інформації про загальну кількість усіх можливих комбінацій між двома наборами даних. Наприклад, розглянемо вигаданий набір даних про учасників навчального процесу та їхній улюблений спосіб навчання.

```
types = np.array([('student', 'offline'), ('student', 'remote'),
                  ('teacher', 'offline'), ('teacher', 'remote'),
                  ('professor', 'offline'), ('professor', 'remote')])
data = types[np.random.choice(range(6), 2000, p=[0.4, 0.1, 0.12, 0.18, 0.05,
   0.15]),:]

# print(data[100:])

label_x, x = np.unique(data[:,0], return_inverse=True)
label_y, y = np.unique(data[:,1], return_inverse=True)
M, xt, yt, _ = plt.hist2d(x,y, bins=(len(label_x), len(label_y)))
plt.xticks((xt[:-1]+xt[1:])/2, label_x)
plt.yticks((yt[:-1]+yt[1:])/2, label_y)
plt.colorbar()
```

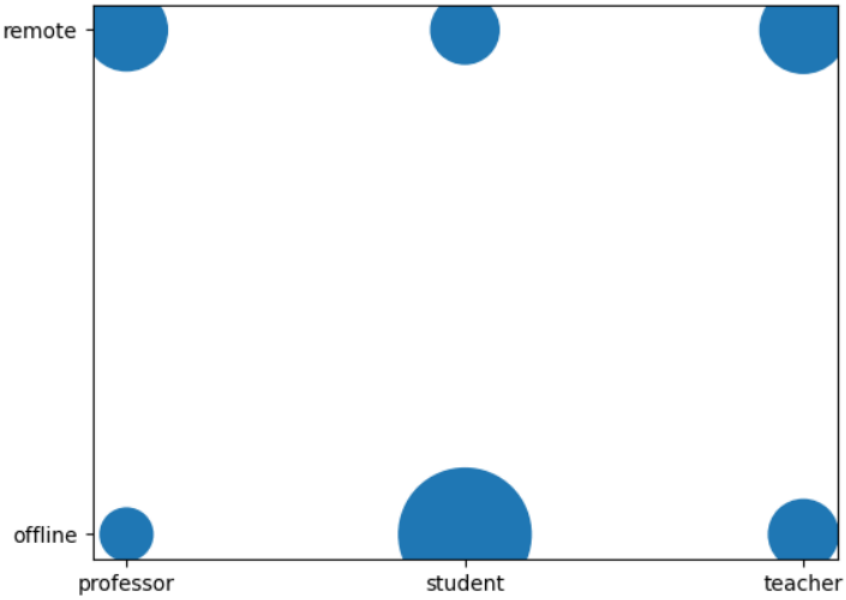
Результат:



Хоча це може бути дещо корисним, діапазон кольорів, безперечно, не є дуже інформативним у деяких налаштуваннях, тому більш доречним може бути діаграма розсіювання з розмірами, пов'язаними з кожним типом даних (це також називається бульбашковою діаграмою).

```
xy, cnts = np.unique((x,y), axis=1, return_counts=True)
plt.scatter(xy[0], xy[1], s=cnts*5)
plt.xticks(range(len(label_x)), label_x)
plt.yticks(range(len(label_y)), label_y)
```

Результат:



Бульбашкова діаграма (bubble plot) додатково використовує розмір маркерів для відображення ще однієї числової змінної. Розмір бульбашок може відображати величину показника, наприклад населення, прибуток, обсяг продажів або іншу числову характеристику.

3.4 3D дані

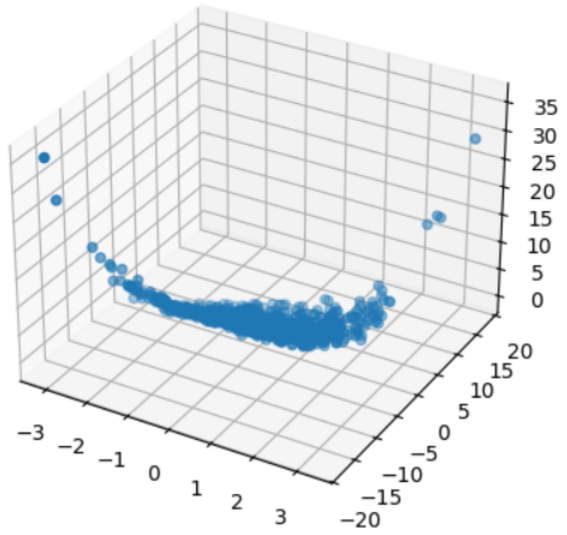
Як тільки ми виходимо за межі двох вимірів, ефективна візуалізація стає набагато складнішою.

3D точкові графіки (3D scatter plots), зображення точок в 3D, слід уникати, якщо, звичайно, не можна взаємодіяти з 3D-графіком:

```
x = np.random.randn(1000)
y = 0.4*x**3 + x + 1.7*np.random.randn(1000)
z = 0.5 + 0.1*(y-1)**2 + 0.4*np.random.randn(1000)

from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(x,y,z)
```

Результат:



3.4.1 Матриці розсіювання (точкові матриці).

Універсальним засобом для візуалізації високорозмірних даних є матриця розсіювання (scatter matrices).

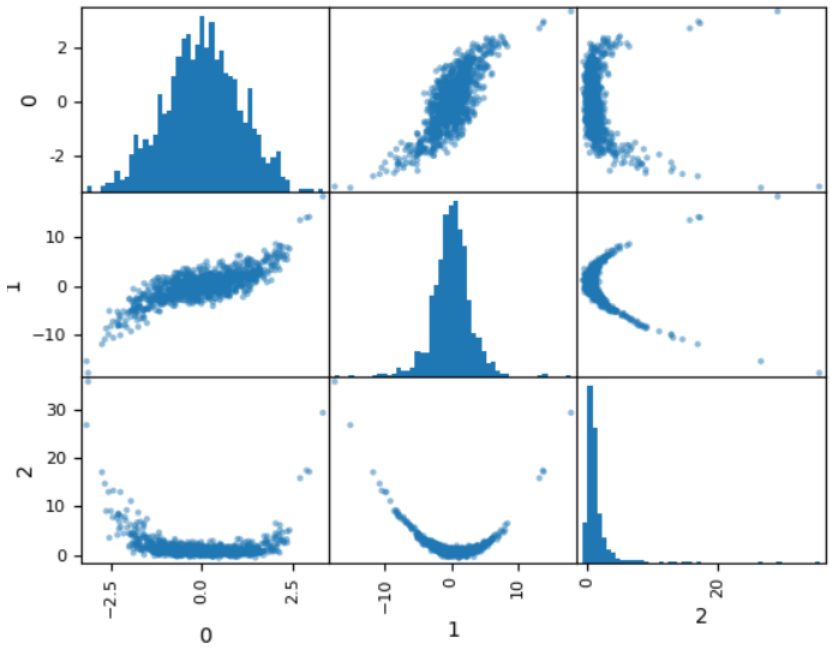
По-суті, це сітка графіків, яка дозволяє візуально оцінити взаємозв'язок між усіма парами змінних у наборі даних. Тобто, для кожного набору двох із трьох аргументів, ми будуємо графік, враховуючи лише відповідні два агрументи, та ігноруючи третій.

☞ Щодо матриць розсіювання слід зазначити один важливий момент: не намагайтеся використовувати їх для представлення даних.

```
import pandas as pd
df = pd.DataFrame([x,y,z]).transpose()
pd.plotting.scatter_matrix(df, hist_kwds={'bins':50})
```

Результат:

```
array([[<Axes: xlabel='0', ylabel='0'>, <Axes: xlabel='1', ylabel='0'>,
        <Axes: xlabel='2', ylabel='0'>],
       [<Axes: xlabel='0', ylabel='1'>, <Axes: xlabel='1', ylabel='1'>,
        <Axes: xlabel='2', ylabel='1'>],
       [<Axes: xlabel='0', ylabel='2'>, <Axes: xlabel='1', ylabel='2'>,
        <Axes: xlabel='2', ylabel='2'>]], dtype=object)
```



Матриця розсіювання використовується для одночасного аналізу взаємозв'язків між кількома числовими ознаками. За допомогою матриці розсіювання можна виявити кореляції, кластери, викиди (outliers) та можливі нелінійні залежності між ознаками.

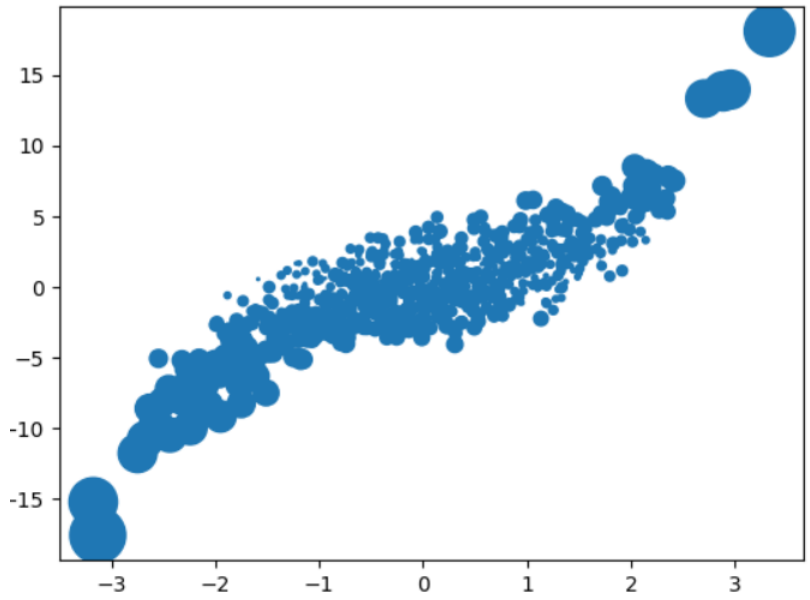
3.4.2 Бульбашкові діаграми.

Бульбашкова діаграма (bubble plots) з попереднього розділу також може бути одним із засобів розуміння тривимірних даних. У цій діаграмі та її варіантах ми побудуємо двовимірну точкову діаграму, але додамо третій вимір за допомогою іншого атрибуту, наприклад розміру точок. Для наведених вище даних це буде виглядати наступним чином.

```
plt.scatter(x,y,s=z*20)
```

Результат:

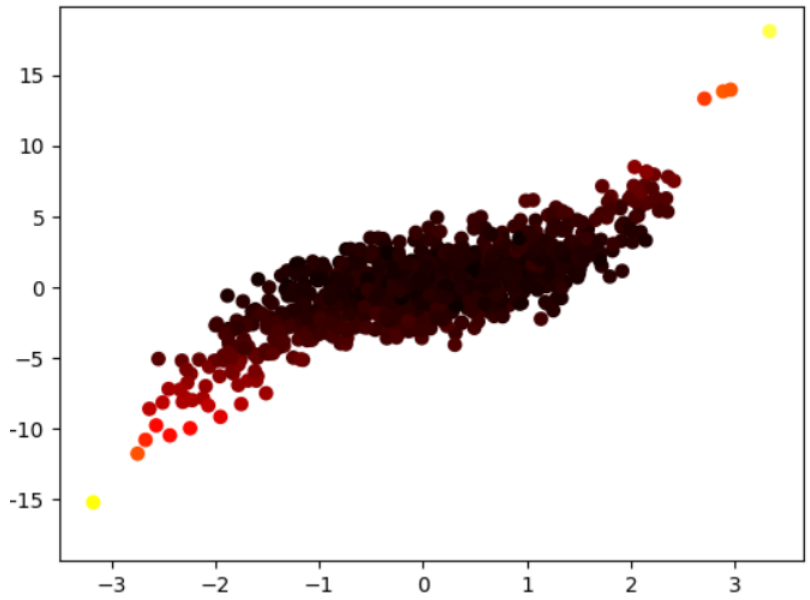
```
<matplotlib.collections.PathCollection at 0x2bfbdcfcd10>
```




```
plt.scatter(x,y,c=z)
```

Результат:

```
<matplotlib.collections.PathCollection at 0x2bfbb610650>
```



Тип діаграми обирають залежно від типу даних, кількості змінних, мети аналізу та закономірностей, які необхідно виявити. Правильно обрана візуалізація робить дані зрозумілішими, дозволяє швидко знаходити закономірності, виявляти аномалії та оцінювати взаємозв'язки між змінними.

✎ **Питання для самоконтролю.**

1. Що таке візуалізація даних і яке її значення в інтелектуальному аналізі даних?
2. Які основні цілі візуалізації даних?
3. Чому візуалізація є важливим етапом дослідницького аналізу даних (EDA)?

4. Які переваги має графічне представлення даних порівняно з табличним?
5. Які бібліотеки Python найчастіше використовуються для побудови графіків?
6. Які типи даних виділяють за шкалою вимірювання?
7. Що таке номінальні дані?
8. Наведіть приклади номінальних даних.
9. Що таке порядкові дані?
10. Наведіть приклади порядкових даних.
11. Що таке інтервальні дані?
12. Чим інтервальні дані відрізняються від даних співвідношення?
13. Що таке дані співвідношення (Ratio Data)?
14. Який тип діаграми найкраще підходить для номінальних даних?
15. Для яких типів даних доцільно використовувати гістограми?
16. Що таке стовпчаста діаграма (Bar Chart)?
17. Для яких задач використовується стовпчаста діаграма?
18. Чим стовпчаста діаграма відрізняється від гістограми?
19. Що таке кругова діаграма (Pie Chart)?
20. У яких випадках доцільно використовувати кругову діаграму?
21. Які недоліки має кругова діаграма?
22. Що таке гістограма?
23. Яку інформацію дозволяє отримати гістограма?
24. Що таке бін (Bin) у гістограмі?
25. Як вибір кількості інтервалів впливає на вигляд гістограми?
26. Що таке точкова діаграма (Scatter Plot)?
27. Для яких задач використовують точкові діаграми?
28. Як за допомогою точкової діаграми можна виявити кореляцію між змінними?
29. Що таке лінійна діаграма (Line Chart)?
30. Для яких типів даних найчастіше використовують лінійні діаграми?
31. Що таке коробкова діаграма (Box Plot)?
32. Які статистичні характеристики відображає коробкова діаграма?
33. Як за допомогою Box Plot можна виявити викиди?
34. Що таке скрипкова діаграма (Violin Plot)?
35. Чим скрипкова діаграма відрізняється від коробкової?

36. Що таке теплова карта (Heatmap)?
37. Які дані найчастіше візуалізують за допомогою теплової карти?
38. Для чого використовується теплова карта кореляцій?
39. Що таке бульбашкова діаграма (Bubble Chart)?
40. Чим бульбашкова діаграма відрізняється від звичайної точкової діаграми?
41. Яку інформацію може відображати розмір бульбашок?
42. Що таке матриця розсіювання (Scatter Matrix)?
43. Для чого використовується матриця розсіювання під час аналізу даних?
44. Які закономірності можна виявити за допомогою матриці розсіювання?
45. Що таке тривимірна (3D) візуалізація даних?
46. Які типи 3D-графіків підтримуються сучасними бібліотеками Python?
47. Які переваги має використання 3D-візуалізації?
48. Які недоліки та обмеження має 3D-візуалізація?
49. За якими критеріями слід обирати тип діаграми для конкретного набору даних?
50. Яким чином правильний вибір способу візуалізації допомагає виявляти закономірності, аномалії та взаємозв'язки у даних?

Розділ 4

Лінійна алгебра та бібліотека NumPy.

Лінійна алгебра — це розділ математики, що вивчає вектори, матриці та операції над ними. Вона є математичною основою більшості алгоритмів машинного навчання, аналізу даних, комп'ютерного зору та нейронних мереж. Методи лінійної алгебри використовуються для представлення даних у вигляді матриць, зменшення розмірності даних, побудови моделей машинного навчання, кластеризації, регресії, обробки зображень та аналізу текстів.

Вектори та матриці відіграють центральну роль в сучасному інтелектуальному аналізі даних. Матриці, це очевидний спосіб зберігання табличних даних. Сьогодні основа лінійної алгебри, є мовою всіх алгоритмів аналізу даних (машинне навчання, штучний інтелект та ін.).

4.1 Основи лінійної алгебри.

Скаляр - окреме числове значення, яке характеризується лише величиною і не має напрямку.
Вектор - 1D масив даних.

$$\mathbf{v} = \begin{bmatrix} x_1 \\ x_2 \\ ... \\ x_n \end{bmatrix}$$

Матриця - 2D масив даних.

$$\mathbf{A} = \begin{bmatrix} x_{11} & x_{12} & ... & x_{1m} \\ x_{21} & x_{22} & ... & x_{2m} \\ ... & ... & ... & ... \\ x_{n1} & x_{n2} & ... & x_{nm} \end{bmatrix}$$

Існують також узагальнення матриць вищого порядку (так звані *тензори*), які представляють тривимірні або вищі масиви значень. Вони досить широко використовуються в сучасній науці про дані, хоча зазвичай (але, звичайно, не завжди) тензори використовуються лише в значенні «багатовимірного масиву», а не в їхньому справжньому лінійно-алгебраїчному значенні. Тензори як лінійні оператори, що діють, наприклад, на матриці або інші тензори вищого порядку, дещо рідше використовуються в базовій науці про дані.

4.1.1 Порядок рядків і стовпців.

Матриці можуть бути розміщені в пам'яті за рядками або за стовпцями. Для того, щоб писати ефективний код для роботи із матричними даними, варто цей розуміти порядок запису.

$$\mathbf{A} = \begin{bmatrix} 100 & 80 \\ 60 & 80 \\ 100 & 100 \end{bmatrix}$$

За рядками: 100, 80, 60, 80, 100, 100. За стовпцями: 100, 60, 100, 80, 80, 100.

☞ По замовчуванню в NumPy порядок за рядками.

4.1.2 Основні операції.

Додавання: Дві матриці A та B, $[n \times m]$, $A + B = C$ визначається: $C_{ij} = A_{ij} + B_{ij}$

Транспонування: Для матриці A, $[n \times m]$, транспонована матриця $C = A^T$ визначається: $C_{ij} = A_{ji}$

Множення: Для матриць A ($[n \times m]$) та B ($[m \times p]$) добуток $C = AB$ ($[n \times p]$) визначається: $C_{ij} = \sum_{k=1}^{m} A_{ik}B_{kj}$

Множення матриць є асоціативним ($(AB)C = A(BC)$), дистрибутивним $A(B+C) = AB+AC$, але не комутативним ($AB \neq BA$).

Одинична матриця: Одинична матриця I це квадратна матриця з одиницями на головній діагоналі та нулями в інших позиціях:

$$\mathbf{I} = \begin{bmatrix} 1 & 0 & ... & 0 \\ 0 & 1 & ... & 0 \\ ... & ... & ... & ... \\ 0 & 0 & ... & 1 \end{bmatrix}$$

Для будь-якої матриці A: $AI = IA = A$

Обернена матриця: Для квадратної матриці A, обернена матриця A^{-1} це така матриця, що

$A^{-1}A = AA^{-1} = I$

Не для кожної квадратної матриці існує обернена.

Деякі рівняння: $(AB)^T = B^T A^T$, $(AB)^{-1} = B^{-1}A^{-1}$

Скалярний добуток векторів: $x * y = x^T y = \sum_{i=1}^{n} x_i y_i$;

Норма вектора: $||x||_2 = \sqrt{\sum_{i=1}^{n} x_i^2}$

4.2 Масиви NumPy.

Створення масиву за допомогою команди `numpy.array` повертає тип `ndarray`. Ви також можете створювати масиви нулів, одиниць або випадкових чисел (у цьому випадку `np.randon.randn` створює матрицю зі стандартними випадковими нормальними елементами, а `np.random.rand` створює рівномірні випадкові елементи).

Спочатку імпортуйте модуль NumPy за допомогою наступної команди:

```
import numpy as np
```

Приклад створення вектора та масиву в NumPy:

```
b = np.array([-13,9])
A = np.array([[4,-5], [-2,3]])
print(b, "\n")
print(A, "\n")
```

Результат:

```
[-13   9]

[[ 4 -5]
 [-2  3]]
```

Приклад 1D масиву одиниць:

```
print(np.ones(40), "\n")
```

Результат:

```
[1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.
 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
```

1D масив нулів:

```
print(np.zeros(4), "\n")
```

Результат:

```
[0. 0. 0. 0.]
```

1D масив сипадкових чисел нормального розподілу $N(0, 1)$:

```
print(np.random.rand(40), "\n")
```

Результат:

```
[0.18507018 0.46626434 0.9801947  0.0827781  0.57378447 0.56871301
 0.84269999 0.6657426  0.56345543 0.93262208 0.98343751 0.30900493
 0.38287176 0.01312199 0.23539386 0.15844975 0.90519996 0.26924736
 0.85669799 0.94183472 0.011728   0.63333182 0.80378177 0.64861789
 0.02308254 0.69805843 0.52183114 0.61711567 0.95150433 0.2085504
 0.76690385 0.1997735  0.45904879 0.81617043 0.62357921 0.8821446
 0.56465777 0.09903507 0.66269635 0.09785717]
```

Приклад 2D масиву одиниць (3×4, 3 рядки 4 стовпчики):

```
print(np.ones((3,4)), "\n")
```

Результат:

```
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]]
```

2D масив нулів (3×4):

```
print(np.zeros((3,4)), "\n")
```

Результат:

```
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
```

2D масив випадкових чисел нормального розподілу:

```
print(np.random.randn(3,4))
```

Результат:

```
[[-0.72206547  0.26200423  1.45715136 -1.25955877]
 [-0.01199544  0.27812632  2.34386974  0.1253467 ]
 [ 1.60420367  1.11023166  1.34434453  0.87136904]]
```

2D масив випадкових чисел нормального розподілу із середнім значенням *mu* та дисперсією *sigma*:

```
sigma, mu = 10, 50
print(sigma * np.random.randn(5, 5) + mu)
```

Результат:

```
[[54.73972213 58.44438959 49.52225911 61.93038812 62.89356473]
 [58.20297378 54.47071892 53.323917   46.94830739 54.82418092]
 [61.0468657  30.48331679 57.33545183 84.93594736 55.92995469]
 [52.76434983 56.28182261 48.24665623 50.28029835 57.08573032]
 [44.30725611 48.51274777 40.88648182 36.13557207 55.81138676]]
```

Ви також можете створити матрицю ідентичності за допомогою команди `np.eye()`, а діагональну матрицю — за допомогою команди `np.diag()`.

Наступний приклад створює масив для одиничної матриці 3×3, та діагональний масив випадкових чисел нормального розподілу:

```
print(np.eye(3),"\n")
print(np.diag(np.random.randn(3)),"\n")
```

Результат:

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]

[[ 0.50651677  0.          0.        ]
 [ 0.         -0.35063699  0.        ]
 [ 0.          0.         -0.72949183]]
```

4.2.1 Індексація в масивах NumPy.

В NumPy можна вибрати окремий елемент масиву, знаючи його положення в масиві, через індексацію `A[i,j]`. Можна також вибрати цілий ряд `A[i,:]`, або зріз масив (slice) `A[start:end, :]`.

```
A = np.array([[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]])
print(A, "\n")
print(A[1,1],"\n")
print(A[1,:],"\n")
print(A[1:3, :], "\n")
```

Результат:

```
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
```

```
 [10 11 12]]

5

[4 5 6]

[[4 5 6]
 [7 8 9]]
```

Можна також вибрати A[1,1] як одиничний 2D масив:

```
print(A[1:2,1:2], "\n")
```

Результат:

```
[[5]]
```

Можна також вибрати окремі рядки, наприклад 1, 2, та 3:

```
print(A[[1,2,3],:], "\n")
```

Результат:

```
[[ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
```

Або, для прикладу, 2, 1, та 2 знову:

```
print(A[[2,1,2],:], "\n")
```

Результат:

```
[[7 8 9]
 [4 5 6]
 [7 8 9]]
```

Можна також вибирати окремі рядки масиву, які нас цікавлять, вказуючи в окремому масиві через bool значення True або False, в залежності від того, чи ми хочемо вибрати відповідний рядок:

```
print(A[[False, True, False, True],:], "\n")
```

Результат:

```
[[ 4  5  6]
 [10 11 12]]
```

4.2.2 Основні операції над масивами в NumPy.

Масиви можна додавати/віднімати, множити/ділити та транспонувати, але ці операції не є ідентичними до відповідних операцій у лінійній алгебрі.

Множення та ділення масивів виконуються поелементно, вони не є множенням матриць або чимось, пов'язаним з оберненням матриць.

Для прикладу, визначимо два масиви:

```
A = np.array([[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]])
B = np.array([[1, 1, 1], [1,2,1], [3, 1, 3], [1, 4, 1]])
```

Додати A та B поелементно:

```
print(A+B, "\n")
```

Результат:

```
[[ 2  3  4]
 [ 5  7  7]
 [10  9 12]
 [11 15 13]]
```

Відняти B від A поелементно:

```
print(A-B, "\n")
```

Результат:

```
[[ 0  1  2]
 [ 3  3  5]
 [ 4  7  6]
 [ 9  7 11]]
```

Поелементне множення (не множення матриць):

```
print(A*B, "\n")
```

Результат:

```
[[ 1  2  3]
 [ 4 10  6]
 [21  8 27]
 [10 44 12]]
```

Поелементне ділення:

```
print(A/B, "\n")
```

Результат:

```
[[ 1.         2.         3.        ]
 [ 4.         2.5        6.        ]
 [ 2.33333333 8.         3.        ]
 [10.         2.75      12.        ]]
```

Ви можете транспонувати масиви, але зверніть увагу, що це має значення тільки для 2D (або вищих) масивів. Транспонування 1D масиву нічого не дає, оскільки NumPy не розрізняє стовпцеві вектори та рядкові вектори для 1D масивів.

```
print(A, "\n")
print(A.T, "\n")
```

Результат:

```
[[ 1  2  3]
 [ 4  5  6]
```

```
 [ 7  8  9]
 [10 11 12]]

[[ 1  4  7 10]
 [ 2  5  8 11]
 [ 3  6  9 12]]
```

4.2.3 Трансляція NumPy.

Справжня цікавість починається, коли ви додаєте/віднімаєте/множите/ділите масиви різних розмірів. Замість того, щоб видавати помилку, NumPy спробує зрозуміти вашу операцію, використовуючи правила трансляції (broadcasting) NumPy. Це складна тема, яка часто бентежить новачків у NumPy, але з невеликою практикою правила стають досить інтуїтивними.

```
A = np.ones((4,3))        # A - 4x3
x = np.array([[1,2,3]])   # x - 1x3

print(A, '\n')
print(x, '\n')
```

Результат:

```
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]

[[1 2 3]]
```

Ми визначили масив 4×3 та вектор розмірності 3.

```
print(A*x)
```

Результат:

```
[[1. 2. 3.]
 [1. 2. 3.]
 [1. 2. 3.]
 [1. 2. 3.]]
```

Коли ми викликали перемноження масиву та вектора, NumPy автоматично повторив x 4 рази, та перемножив на масив.

4.2.4 Операції лінійної алгебри в NumPy.

Починаючи з Python 3, між масивами NumPy тепер існує оператор множення матриць `@` (раніше для цього доводилося використовувати більш громіздку функцію `np.dot()`).

```
A = np.random.randn(5,4)
C = np.random.randn(4,3)
x = np.random.randn(4)
y = np.random.randn(5)
z = np.random.randn(4)
```

Приклад множення матриці на матрицю (вертає 2D масив):

```
print(A @ C, "\n")
```

Результат:

```
[[-0.77983164  0.86915701  0.74016041]
 [ 0.9441811  -1.13337787 -2.45126146]
 [ 1.06065113 -3.17950771 -3.50084458]
 [ 1.26732128 -1.3041138  -2.5499486 ]
 [ 1.60891994 -4.68541868 -4.6433185 ]]
```

Приклад множення матриці на вектор (вертає 1D масив):

```
print(A @ x, "\n")
```

Результат:

```
[ 1.53587182 -2.9122003  -2.53401694 -2.88846645 -3.50775768]
```

Внутрішній добуток (скалярний) - зверніть увагу на тип даних, що вертаються - це скаляр, а не масив!

```
print(x @ z)
```

Результат:

```
2.236322572387176
```

☞ За замовчуванням оператор `@` застосовується зліва направо, що може призвести до дуже неефективних порядків множення матриць.

```
A = np.random.randn(1000,1000)
B = np.random.randn(1000,2000)
x = np.random.randn(2000)

%timeit A @ B @ x
```

Результат:

```
59.1 ms +/- 7.45 ms per loop (mean +/- std. dev. of 7 runs, 10 loops each)
```

Це виконує матричні добутки $(AB)x$, які спочатку обчислюють неефективне множення матриць. Якщо ми хочемо обчислити добуток у набагато ефективнішому (в даному прикладі) порядку $A(Bx)$, ми вказуємо порядок виконання операції множення матриць через дужки:

```
%timeit A @ (B @ x)
```

Результат:

```
2.13 ms +/- 226' ns per loop (mean +/- std. dev. of 7 runs, 100 loops each)
```

Нарешті, Numpy включає процедуру `np.linalg.inv()` для обчислення оберненої матриці A^{-1} та `np.linalg.solve()` для обчислення матричного рівняння $A^{-1}b$.

```
b = np.array([-13,9])
A = np.array([[4,-5], [-2,3]])
```

```
print(np.linalg.inv(A), "\n")
print(np.linalg.solve(A,b))
```

Результат:

```
[[1.5 2.5]
 [1.  2. ]]

[3. 5.]
```

4.2.5 Складність операцій.

Припустимо, що A, B $[n \times n]$, x, y - вектори [n].
Добуток матриць AB: $O(n^3)$
Добуток матриці та вектора Ax: $O(n^2)$
Внутрішній добуток векторів $x^T y$: $O(n)$
Обернена матриця/розв'язання: A^{-1}: $O(n^3)$

4.2.6 Розріджені матриці.

Багато матриць є розрідженими - такими, що містять переважно нульові елементи, з лише декількома ненульовими елементами.

Приклади: матриці, утворені реальними графами, матриці підрахунку слів у документах (більше про це пізніше)

Зберігання всіх цих нулів у стандартному форматі матриці може бути великою тратою обчислювальних ресурсів і пам'яті.

Бібліотеки розріджених матриць надають ефективні засоби для обробки цих розріджених матриць, зберігаючи і оперуючи тільки ненульовими елементами

Існує кілька різних способів зберігання розріджених матриць, кожен з яких оптимізований для різних операцій

Формат координат (COO): зберігання кожного елемента як кортежу ($inde_{row}$, $index_{column}$, $value$)

Наприклад, замість матриці:

$$\mathbf{A} = \begin{bmatrix} 0 & 0 & 3 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 4 & 0 & 1 & 0 \end{bmatrix}$$

Матимемо:

$$values = [2, 4, 1, 3, 1, 1]$$

$$indices_{row} = [1, 3, 2, 0, 3, 1]$$

$$indecies_{column} = [0, 0, 1, 2, 2, 3]$$

Приклад використання біблотеки `scipy` для роботи з розрідженими матрицями:

```
import scipy.sparse as sp

values = [2, 4, 1, 3, 1, 1]
row_indices = [1, 3, 2, 0, 3, 1]
column_indices = [0, 0, 1, 2, 2, 3]
A = sp.coo_matrix((values, (row_indices, column_indices)), shape=(4,4))
print(A.todense())
```

Результат:

```
[[0 0 3 0]
 [2 0 0 1]
 [0 1 0 0]
 [4 0 1 0]]
```

✎ **Питання для самоконтролю.**

1. Що таке лінійна алгебра і чому вона є важливою для інтелектуального аналізу даних?
2. Які задачі аналізу даних розв'язуються за допомогою методів лінійної алгебри?
3. Що таке скаляр?
4. Що таке вектор?
5. Що таке матриця?
6. Чим відрізняється вектор від матриці?
7. Які основні операції можна виконувати над векторами?
8. Які основні операції можна виконувати над матрицями?
9. Що таке транспонування матриці?
10. Що таке одинична матриця?
11. Що таке нульова матриця?
12. Що таке квадратна матриця?
13. Які умови повинні виконуватися для множення двох матриць?
14. Що таке скалярний добуток векторів?
15. Для чого використовується матричне множення в машинному навчанні?
16. Що таке визначник (детермінант) матриці?
17. Для чого використовується обернена матриця?
18. Що таке власні значення та власні вектори матриці?
19. Чому власні значення важливі для методів аналізу даних?
20. Що таке бібліотека NumPy?
21. Які переваги NumPy порівняно зі звичайними списками Python?
22. Як встановити бібліотеку NumPy?

23. Як імпортувати NumPy у Python-програму?
24. Що таке масив (ndarray) у NumPy?
25. Як створити одновимірний масив NumPy?
26. Як створити двовимірний масив NumPy?
27. Як отримати розмірність масиву?
28. Як визначити форму (shape) масиву?
29. Як створити масив, заповнений нулями?
30. Як створити масив, заповнений одиницями?
31. Для чого використовуються функції arange() та linspace()?
32. Як змінити форму масиву за допомогою reshape()?
33. Як звернутися до окремого елемента масиву?
34. Як виконати зрізи (slicing) масивів?
35. Як працює індексація багатовимірних масивів?
36. Як виконуються арифметичні операції над масивами NumPy?
37. Що таке векторизовані обчислення?
38. Які переваги векторизації над використанням циклів?
39. Як обчислити суму, середнє значення та стандартне відхилення елементів масиву?
40. Як знайти максимальне та мінімальне значення масиву?
41. Як виконати множення матриць у NumPy?
42. Для чого використовується функція dot()?
43. Як знайти транспоновану матрицю в NumPy?
44. Як обчислити обернену матрицю та визначник за допомогою NumPy?
45. Яку роль відіграють NumPy та лінійна алгебра в сучасному машинному навчанні та інтелектуальному аналізі даних?

Розділ 5

Обробка Природної Мови.

Велика кількість даних у багатьох реальних наборах даних представлена у вигляді так званого вільного тексту (free text). Це є, наприклад, коментарі користувачів, телеметрія програмних додатків, будь-які «неструктуровані» поля, та багато чого іншого.

Обчислювальна обробка природної мови полягає в тому, що ми пишемо комп'ютерні програми, які можуть розуміти природну мову. Обробка природної мови (NLP) займається тим, як перетворити текст — часто нечіткий і непідготовлений — у форму, зручну для аналізу та моделювання. В цьому розділі ми спробуємо отримати деяку значущу інформацію з неструктурованих текстових даних.

5.1 Попередня обробка тексту.

5.1.1 Очищення тексту.

Першим кроком до роботи з такими даними зазвичай стає підготовка тексту: приведення символів до одного регістру, видалення розділових знаків і зайвих символів, очищення чисел і зведення виробленого шуму у більш стандартизований вигляд.

Застосуємо біблотеку nltk для цих цілей.

```
import re
import string
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import nltk
```

Спершу завантажимо ресурси для роботи із найпоширенішими словами в англійській мові ('the', 'is', 'and') та для роботи із пунктуацією: "stopwords"та "punkt":

```
nltk.download('stopwords')
nltk.download('punkt')
```

Перш за все, варто розпочати роботу з текстом із того, щоб перевести всі літери в нижній регістр:

```
text = text.lower()
```

Та видалити розділові знаки:

```
text = text.translate(str.maketrans('', '', string.punctuation))
```

Також із тексту можна видалити числа:

```
text = re.sub(r'\d+', '', text)
```

Та спеціальні символи:

```
text = re.sub(r'[^a-z\s]', '', text)
```

Щоб видалити найпоширеніші в англійській мові стоп-слова:

```
stop_words = set(stopwords.words('english'))
text = ' '.join(word for word in word_tokenize(text) if word not in stop_words)
```

Повністю очищення тексту виглядає наступним чином:

```
import re
import string
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import nltk

nltk.download('stopwords')
nltk.download('punkt')

def clean_text(text):
        text = text.lower()

        text = text.translate(str.maketrans('', '', string.punctuation))

        text = re.sub(r'\d+', '', text)

        text = re.sub(r'[^a-z\s]', '', text)

        stop_words = set(stopwords.words('english'))
        text = ' '.join(word for word in word_tokenize(text) if word not in
           stop_words)

        return text

sample_text = "Hello! This is an example text with numbers 123 and punctuation!!!"
cleaned_text = clean_text(sample_text)
print(cleaned_text)
```

Результат:

```
hello example text numbers punctuation
```

5.1.2 Токенізація, стемінг, лематизація, нормалізація.

Після базового очищення йде токенізація — розбиття тексту на слова або речення, що дозволяє оперувати окремими «будівельними блоками» змісту. Далі часто застосовують стемінг

або лематизацію: стемінг обрізає закінчення й повертає корінь слова, іноді утворюючи несловникові форми, тоді як лематизація намагається відновити словникову форму з урахуванням контексту. Нормалізація може також включати виправлення орфографії, розгортання абревіатур і узгодження синонімів — усе задля того, щоб одна й та сама ідея не розпадалася на багато еквівалентних представлень.

Приклад токенізації:

```
from nltk.tokenize import word_tokenize, sent_tokenize

def tokenize_text(text):
        words = word_tokenize(text)
        sentences = sent_tokenize(text)
        return words, sentences

sample_text = "Natural Language Processing (NLP) is exciting! It enables computers
    to understand human language."
words, sentences = tokenize_text(sample_text)
print('Words:', words)
print('Sentences:', sentences)
```

Результат:

```
Words: ['Natural', 'Language', 'Processing', '(', 'NLP', ')', 'is', 'exciting', '!', 'It', 'enables',
    'computers', 'to', 'understand', 'human', 'language', '.']
Sentences: ['Natural Language Processing (NLP) is exciting!', 'It enables computers to understand human
    language.']
```

Перш ніж перейти до лематизації, варто завантажити в nltk спеціальний словник для цієї потреби:

```
nltk.download('wordnet')
```

Стемінг та лематизація:

```
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.tokenize import word_tokenize
import nltk

nltk.download('wordnet')

def apply_stemming(text):
        stemmer = PorterStemmer()
        words = word_tokenize(text)
        stemmed_words = [stemmer.stem(word) for word in words]
        return stemmed_words

def apply_lemmatization(text):
        lemmatizer = WordNetLemmatizer()
        words = word_tokenize(text)
        lemmatized_words = [lemmatizer.lemmatize(word) for word in words]
        return lemmatized_words
```

```
sample_text = "The children are playing with their toys."
print('Stemmed:', apply_stemming(sample_text))
print('Lemmatized:', apply_lemmatization(sample_text))
```

Результат:

```
Stemmed: ['the', 'children', 'are', 'play', 'with', 'their', 'toy', '.']
Lemmatized: ['The', 'child', 'are', 'playing', 'with', 'their', 'toy', '.']
```

Приклад нормалізації з використанням модуля autocorrect:

```
from nltk.tokenize import word_tokenize
from autocorrect import Speller
import nltk

nltk.download('punkt')

def normalize_text(text):
        spell = Speller(lang='en')
        normalization_dict = {
                "u": "you",
                "ur": "your",
                "r": "are",
                "pls": "please",
                "thx": "thanks"
        }
        words = word_tokenize(text.lower())
        normalized_words = [normalization_dict.get(word, spell(word)) for word in
            words]
        normalized_text = ' '.join(normalized_words)
        normalized_text = normalized_text.replace('!', '').replace('.', '').strip()
        return normalized_text

sample_text = "U r the best! Pls help me with this assignment. Thx!"
print(normalize_text(sample_text))
```

Результат:

```
you are the best  please help me with this assignment  thanks
```

5.2 Чисельне представлення тексту.

Розуміння мови є складним завданням.

Розглянемо такий текст: «Під час полювання в Африці я застрелив слона в піжамі. Як він потрапив у мою піжаму, я не знаю». Без повного бачення всього тексту тяжко зрозуміти, що саме мається на увазі, наприклад, в першому реченні.

Основна думка полягає в наступному: ми використовуємо неймовірну кількість контексту, щоб зрозуміти значення речень природної мови. Саме з розумінням цього контексту допомагають великі мовні моделі та механізм attention в трансформерній архітектурі. Але чи завжди це настільки складно, щоб вимагати застосування передових технологій обробки тексту?

Розглянемо дві рецензії на фільм:

*"Barbie is a fun movie that has surprisingly plenty of **heartwarming** moments and **good** messages to offer. The set designs are **eye-catching** and on point as well. Both Margot Robbie and Ryan Gosling nailed their parts, but it is Gosling who mostly shines."*

*"It didn't grab me; I didn't find it funny anywhere. It went from the most **annoying** opening to as far as one can get annoying in every second of this film. The music was **shallow**; the production design teetered on Terry Gilliam's work but without the imagination or talent, and old 1920s/1930s film backgrounds and old Three Stooges routines were very **tiring**."*

Який з них є позитивним? Ми можемо уце зрозуміти, просто глянувши на виділені слова. Часто ми можемо дуже легко визначити «загальний зміст» тексту природною мовою, навіть не розуміючи речень.

5.2.1 Мішок слів.

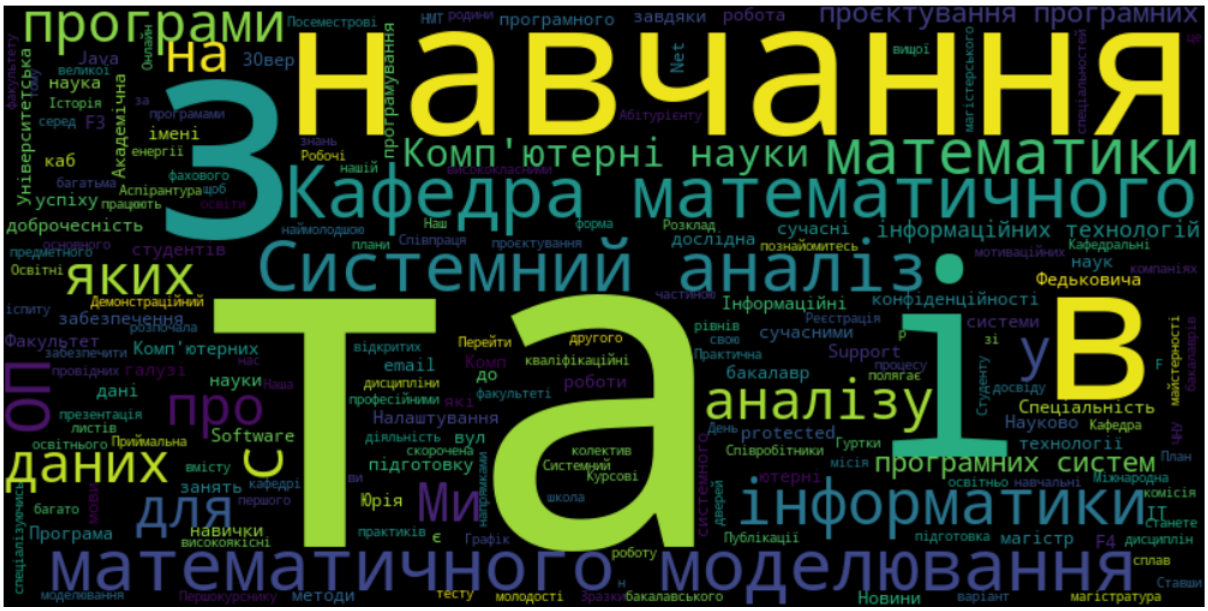
Після підготовки тексту його потрібно представити чисельно. Простий і часто ефективний підхід — модель «мішка слів», при якій кожен документ кодується вектором частот слів.

Генерація wordcloud (приклад збору тексту зі сторінки та візуалізації):

```
from wordcloud import WordCloud
from bs4 import BeautifulSoup
import requests
import re

response = requests.get("https://mathmod.chnu.edu.ua/")
root = BeautifulSoup(response.content, "lxml")
wc = WordCloud(width=800, height=400).generate(re.sub(r"\s+", " ", root.text))
wc.to_image()
```

Результат:



5.2.2 TFIDF та косинусна подібність тексту.

«Частота терміна» просто означає кількість кожного слова в документі. Позначається $tf_{i,j}$ частотою слова i в документі (тексті) j (іноді індекси обертаються, ми використовуємо їх для

узгодженості з матрицею).

```
documents = [
    "In Inception, the plot unfolds with thrilling action, keeping audiences on
       edge through a series of intense and unexpected twists.", # t_i_0 = 1
    "The story of Inception keeps audiences hooked with its thrilling action and
       surprising twists, making every moment unforgettable.", # t_i_1 = 1
    "The Matrix transformation preserves the vector's direction, illustrating how
       linear combinations affect the overall system." # t_i_2 = 0
]

document_words = [doc.lower().split() for doc in documents]
vocab = sorted(set(sum(document_words, [])))
vocab_dict = {k:i for i,k in enumerate(vocab)}
print(vocab, "\n")
print(vocab_dict, "\n")
```

Результат:

```
['a', 'action', 'action,', 'affect', 'and', 'audiences', 'combinations', 'direction,', 'edge', 'every',
    'hooked', 'how', 'illustrating', 'in', 'inception', 'inception,', 'intense', 'its', 'keeping', 'keeps',
    'linear', 'making', 'matrix', 'moment', 'of', 'on', 'overall', 'plot', 'preserves', 'series', 'story',
    'surprising', 'system.', 'the', 'thrilling', 'through', 'transformation', 'twists,', 'twists.',
    'unexpected', 'unfolds', 'unforgettable.', "vector's", 'with']

{'a': 0, 'action': 1, 'action,': 2, 'affect': 3, 'and': 4, 'audiences': 5, 'combinations': 6, 'direction,':
    7, 'edge': 8, 'every': 9, 'hooked': 10, 'how': 11, 'illustrating': 12, 'in': 13, 'inception': 14,
    'inception,': 15, 'intense': 16, 'its': 17, 'keeping': 18, 'keeps': 19, 'linear': 20, 'making': 21,
    'matrix': 22, 'moment': 23, 'of': 24, 'on': 25, 'overall': 26, 'plot': 27, 'preserves': 28, 'series':
    29, 'story': 30, 'surprising': 31, 'system.': 32, 'the': 33, 'thrilling': 34, 'through': 35,
    'transformation': 36, 'twists,': 37, 'twists.': 38, 'unexpected': 39, 'unfolds': 40, 'unforgettable.':
    41, "vector's": 42, 'with': 43}
```

Тепер побудуємо матрицю, яка містить кількість слів (частоту вживання термінів) для всіх документів.

```
import numpy as np

X_tf = np.zeros((len(documents), len(vocab)), dtype=int)
for i, doc in enumerate(document_words):
    for word in doc:
        X_tf[i, vocab_dict[word]] +=1

print(X_tf)
```

Результат:

```
[[1 0 1 0 1 1 0 0 1 0 0 0 0 1 0 1 1 0 1 0 0 0 0 0 1 1 0 1 0 1 0 0 0 1 1 1
  0 0 1 1 1 0 0 1]
 [0 1 0 0 1 1 0 0 0 1 1 0 0 0 1 0 0 1 0 1 0 1 0 1 1 0 0 0 0 0 1 1 0 1 1 0
  0 1 0 0 0 1 0 1]
 [0 0 0 1 0 0 1 1 0 0 0 1 1 0 0 0 0 0 0 0 1 0 1 0 0 0 1 0 1 0 0 0 1 3 0 0
  1 0 0 0 0 0 1 0]]
```

Частота вживання термінів, як правило, «перевантажена» дуже поширеними словами («the»,

«is», «of» тощо).

☞ Примітка: дуже поширені слова, які називаються «стоп-словами», можуть бути навіть видалені з використанням засобів nltk, як ми це обговорювали на початку цього розділа.

Ідея зворотної частоти документа полягає в тому, щоб надавати словам негативну вагу пропорційно до частоти їх вживання в усьому наборі документів.

$$idf_i = \log(\frac{N_{documents}}{N_{documents-with-word-i}})$$

Як і у випадку з частотою терміна, існують також інші версії з різними масштабами, але найпоширенішим є вищезазначений логарифмічний масштаб.

Зверніть увагу, що обернена частота документа визначається лише для слів, а не для пар «слово-документ», як у випадку з частотою терміна.

```
idf = np.log(X_tf.shape[0]/X_tf.astype(bool).sum(axis=0))
print(idf)
```

Результат:

```
[1.09861229 1.09861229 1.09861229 1.09861229 0.40546511 0.40546511
 1.09861229 1.09861229 1.09861229 1.09861229 1.09861229 1.09861229
 1.09861229 1.09861229 1.09861229 1.09861229 1.09861229 1.09861229
 1.09861229 1.09861229 1.09861229 1.09861229 1.09861229 1.09861229
 0.40546511 1.09861229 1.09861229 1.09861229 1.09861229 1.09861229
 1.09861229 1.09861229 1.09861229 0.         0.40546511 1.09861229
 1.09861229 1.09861229 1.09861229 1.09861229 1.09861229 1.09861229
 1.09861229 0.40546511]
```

Зрештою, поєднання частоти терміна та зворотної частоти документа дає відому міру TF–IDF (term frequency inverse document frequency):

$$tfidf_{i,j} = tf_{i,j} \times idf_i,$$

яка підсилює рідкісні й інформативні слова й послаблює надто загальні.

```
X_tfidf = X_tf * idf
print(X_tfidf)
```

Результат:

```
[[1.09861229 0.         1.09861229 0.         0.40546511 0.40546511
  0.         0.         1.09861229 0.         0.         0.
  0.         1.09861229 0.         1.09861229 1.09861229 0.
  1.09861229 0.         0.         0.         0.         0.
  0.40546511 1.09861229 0.         1.09861229 0.         1.09861229
  0.         0.         0.         0.         0.40546511 1.09861229
  0.         0.         1.09861229 1.09861229 1.09861229 0.
  0.         0.40546511]
 [0.         1.09861229 0.         0.         0.40546511 0.40546511
  0.         0.         0.         1.09861229 1.09861229 0.
  0.         0.         1.09861229 0.         0.         1.09861229
  0.         1.09861229 0.         1.09861229 0.         1.09861229
  0.40546511 0.         0.         0.         0.         0.
  1.09861229 1.09861229 0.         0.         0.40546511 0.
  0.         1.09861229 0.         0.         0.         1.09861229
  0.         0.40546511]
 [0.         0.         0.         1.09861229 0.         0.
  1.09861229 1.09861229 0.         0.         0.         1.09861229
```

```
  1.09861229 0.         0.         0.         0.
  0.         0.         1.09861229 0.         1.09861229 0.
  0.         0.         1.09861229 0.         1.09861229 0.
  0.         0.         1.09861229 0.         0.         0.
  1.09861229 0.         0.         0.         0.
  1.09861229 0.        ]]
```

Косинусна подібність — простий спосіб виміряти схожість між двома векторними представленнями тексту:

$$CosineSim(x, y) = \frac{x \cdot y}{\|x\|_2 \|y\|_2}.$$

Більші числа означають, що вектори tfidf є ближчими в n-вимірному просторі, а отже, тексти подібніші між собою.

```
X_tfidf_norm = X_tfidf / np.linalg.norm(X_tfidf, axis=1)[:,None]
M = X_tfidf_norm @ X_tfidf_norm.T
print(M)
```

Результат:

```
[[1.         0.04991504 0.        ]
 [0.04991504 1.         0.        ]
 [0.         0.         1.        ]]
```

Як бачимо, тексти, що описували одні і ті ж речі, отримали більшу косинусну подібність, а третій текст з обома іншими має косинусну подібність 0.

Використання tfidf та косинусної подібності повністю:

```
import numpy as np

documents = [
        "In Inception, the plot unfolds with thrilling action, keeping audiences on
           edge through a series of intense and unexpected twists.",
        "The story of Inception keeps audiences hooked with its thrilling action
           and surprising twists, making every moment unforgettable.",
        "The Matrix transformation preserves the vector's direction, illustrating
           how linear combinations affect the overall system."
]

document_words = [doc.lower().split() for doc in documents]
vocab = sorted(set(sum(document_words, [])))
vocab_dict = {k: i for i, k in enumerate(vocab)}

X_tf = np.zeros((len(documents), len(vocab)), dtype=int)
for i, doc in enumerate(document_words):
        for word in doc:
                X_tf[i, vocab_dict[word]] += 1


idf = np.log(X_tf.shape[0] / X_tf.astype(bool).sum(axis=0))
X_tfidf = X_tf * idf
X_tfidf_norm = X_tfidf / np.linalg.norm(X_tfidf, axis=1)[:, None]
```

```
M = X_tfidf_norm @ X_tfidf_norm.T
print('Cosine similarity matrix:\n', M)
```

Результат:

```
Cosine similarity matrix:
 [[1.         0.04991504 0.        ]
 [0.04991504 1.         0.        ]
 [0.         0.         1.        ]]
```

☞ Косинусна подібність добре працює для порівняння документів у просторі TF–IDF, але має й обмеження: в такому підході немає розуміння семантики слів, і різні слова, які мають близьке значення, залишаються віддаленими в просторі one-hot або частотних векторів.

Наприклад, наступні окремі слова мають між собою відстань 1, хоча деякі із них є більш спорідненими між собою:

$$||kyiv - zhytomyr|| = ||zhytomyr - uncle|| = 1$$

5.2.3 Векторне подання слів.

Щоб виправити недоліки косинусної подібності, використовують вбудовування слів (word embeddings) — щільні вектори фіксованого розміру, у яких семантично близькі слова займають близькі позиції у просторі. Моделі на кшталт word2vec або переднавчені матриці типу GloVe навчають векторні представлення таким чином, що арифметичні операції над векторами можуть відображати мовні відношення: наприклад, операція king − man + woman зазвичай наближається до вектора, близького до queen. Це відкриває шлях до більш змістовних порівнянь і до знаходження синонімів, антонімів і тематичних зв'язків.

Щоб завантажити векторне подання, варто спочатку встановити модуль sentence-transformers, та завантажити претреновану трансофрмер модель:

```
import numpy as np
from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer('all-MiniLM-L6-v2')
```

Давайте глянемо, як виглядає векторне подання слова "computer"(або, хоча б, перші 10 чисел):

```
vector = model.encode(['computer'])[0]
print('vector sample:', vector[:10])
```

Результат:

```
vector sample: [-0.05127361  0.04783836 -0.00333908 -0.0388703  -0.03269714 -0.0328583
  0.09360427  0.03932324  0.08196709  0.01824838]
```

Тепер ми можемо обчислювати подібності між окремими словами:

```
embeddings = model.encode(['woman', 'man', 'kyiv'])

similarity = util.cos_sim(embeddings[0], embeddings[1]).item()
print('similarity woman/man:', similarity)
```

```
similarity = util.cos_sim(embeddings[1], embeddings[2]).item()
print('similarity man/kyiv:', similarity)
```

Результат:

```
similarity woman/man: 0.32567834854125977
similarity man/kyiv: 0.17196238040924072
```

Тепер, для прикладу, обчислимо вектор 'King - Man + Woman' та знайдемо схожі до нього.

```
words_to_encode = ['king', 'man', 'woman']
word_vectors = model.encode(words_to_encode)

king = word_vectors[0]
man = word_vectors[1]
woman = word_vectors[2]

result_vector = king - man + woman

vocabulary = ['queen', 'princess', 'monarch', 'empress', 'man', 'computer',
    'throne', 'kyiv']
vocab_embeddings = model.encode(vocabulary)

similarities = util.cos_sim(result_vector, vocab_embeddings)[0].numpy()

top_indices = np.argsort(similarities)[::-1][:5]

print('\nAnalogy results:')
for idx in top_indices:
    print(f"{vocabulary[idx]}: {similarities[idx]:.4f}")
```

Результат:

```
Analogy results:
queen: 0.5795
monarch: 0.5469
throne: 0.4804
princess: 0.4418
empress: 0.4368
```

Як бачимо, вектор 'King - Man + Woman' доволі таки близький до вектора 'Queen'.

Проведемо тепер tfidf-аналіз та знайдемо косинусну подібність між текстами, використовуючи тепер векторне представлення кожного окремого слова.

```
from sentence_transformers import SentenceTransformer, util

documents = [
    "kyiv has some excellent new restaurants",
    "kharkiv is a city with great cuisine",
    "postgresql is a relational database management system"
]

model = SentenceTransformer('all-MiniLM-L6-v2')
```

```
X_embeddings = model.encode(documents)
print("Embedding matrix:")
print(X_embeddings)
similarity_matrix = util.cos_sim(X_embeddings, X_embeddings)

print("\nCosinus similarity matrix:")
print(similarity_matrix.numpy())
```

Результат:

```
Embedding matrix:
[[-0.00245756 -0.05211234  0.05479724 ...  0.05498891 -0.00664046
  -0.02283956]
 [ 0.04663361  0.06056985 -0.02163695 ...  0.04387321 -0.0416828
   0.0089187 ]
 [ 0.01276393 -0.00399948 -0.08929092 ... -0.02673415  0.04939254
   0.0399594 ]]

Cosinus similarity matrix:
[[ 1.          0.53684413 -0.01055246]
 [ 0.53684413  1.          0.03759826]
 [-0.01055246  0.03759826  1.        ]]
```

Далі стоять мовні моделі, які намагаються оцінити ймовірність слова, враховуючи контекст: $P(word_i \mid word_1, \ldots, word_{i-1})$. Простий, але корисний клас таких моделей — n-грами, що враховують обмежений контекст попередніх $n-1$ слів; їх можна вважати аппроксимацією повної ймовірнісної моделі мови. Сучасні нейронні та трансформерні моделі значно перевершують n-грами за здатністю захоплювати довготривалі залежності та контекст.

У практиці науковця з даних частина роботи полягає в доборі правильного представлення залежно від задачі: для швидкої класифікації рецензій часто достатньо TF–IDF і лінійних моделей; для складних завдань семантичного пошуку або витягнення сутностей корисні вбудовування слів і контекстні мовні моделі. Важливо пам'ятати, що підготовка даних і прості методи іноді дають вражаючі результати, і вибір інструменту має базуватись на доступності даних, вимогах до точності й обмеженнях по часу на навчання.

✎ **Питання для самоконтролю.**

1. Що таке обробка природної мови (Natural Language Processing, NLP)?
2. Які задачі належать до сфери обробки природної мови?
3. Чому текстові дані потребують попередньої обробки?
4. Які основні етапи попередньої обробки тексту?
5. Що таке токенізація?
6. Які існують способи токенізації тексту?
7. Для чого виконується очищення тексту?
8. Які символи зазвичай видаляють під час очищення тексту?
9. Що таке стоп-слова?
10. Чому стоп-слова часто вилучають із тексту?
11. У яких випадках видаляти стоп-слова недоцільно?

12. Що таке нормалізація тексту?
13. Для чого текст переводять до нижнього регістру?
14. Чим нормалізація відрізняється від токенізації?
15. Що таке стемінг?
16. Які задачі вирішує стемінг?
17. Які переваги та недоліки стемінгу?
18. Що таке лематизація?
19. Чим лематизація відрізняється від стемінгу?
20. У яких випадках доцільніше використовувати лематизацію?
21. Що таке хмара слів (Word Cloud)?
22. Для чого використовується хмара слів?
23. Які обмеження має хмара слів як інструмент аналізу?
24. Що таке модель «мішок слів» (Bag of Words)?
25. Як формується матриця Bag of Words?
26. Які недоліки моделі Bag of Words?
27. Що таке TF (Term Frequency)?
28. Як обчислюється TF?
29. Що таке IDF (Inverse Document Frequency)?
30. Чому рідкісні слова мають вищу вагу в IDF?
31. Що таке TF-IDF?
32. Які переваги TF-IDF порівняно з Bag of Words?
33. У яких задачах застосовується TF-IDF?
34. Що таке векторне подання тексту?
35. Як текст перетворюється на числовий вектор?
36. Що таке косинусна подібність?
37. Як інтерпретується значення косинусної подібності?
38. Чому косинусна подібність є популярною мірою схожості документів?
39. Що таке векторне подання слів (Word Embeddings)?
40. Чим векторні подання слів відрізняються від TF-IDF?
41. Які переваги мають моделі Word2Vec та GloVe?
42. Які сучасні моделі контекстних векторних подань слів існують?
43. У яких задачах використовуються векторні подання слів?
44. Які бібліотеки Python найчастіше застосовують для обробки природної мови?
45. Які сучасні напрями розвитку технологій обробки природної мови пов'язані з великими мовними моделями (LLM)?

Розділ 6

Теорія ймовірності в контексті аналізу даних.

6.1 Основи теорії ймовірності.

Теорія ймовірності це розділ математики, що вивчає закономірності випадкових явищ і дає змогу оцінити, наскільки ймовірною є та чи інша подія.

Ніколи не можна точно знати, якою, наприклад, буде завтрашня погода. Проте маючи дані спостереження за регіоном на протязі певного періоду часу в даний сезон, можна оцінити, з якою ймовірністю завтра буде дощ. Спеціаліст з аналізу даних повинен вміти працювати з ймовірностями для того, щоб працювати з невизначеністю, прогнозувати майбутні події, оцінювати ризики та будувати моделі машинного навчання.

Теорія ймовірності лежить в основі багатьох алгоритмів машинного навчання, статистики, прогнозування часових рядів, аналізу ризиків і систем підтримки прийняття рішень.

```
import random
import math
import numpy as np
from collections import Counter

pX = {1: 0.1, 2: 0.5, 3:0.4}
pWeather = {"sunny": 0.3, "rainy":0.2, "cloudy":0.4, "snowy":0.1}
print(pWeather["sunny"])
```

Результат:

```
0.3
```

Приклад: Кидання монети. Симулюємо 10 підкидувань, обчислюємо емпіричну ймовірність випадання «решки».

```
tosses = [random.choice(["H", "T"]) for _ in range(10)]
print(tosses)

p_heads = tosses.count("H") / len(tosses)
print(f"P(Heads) approx. {p_heads:.2f}")
```

Результат:

```
['H', 'T', 'T', 'H', 'H', 'H', 'H', 'T', 'T', 'T']
P(Heads) approx. 0.50
```

6.1.1 Закон великих чисел

Давайте подивимося, як емпірична ймовірність наближається до теоретичної ймовірності.

```
def simulate_coin(n_tosses=1000):
    tosses = [random.choice(["H", "T"]) for _ in range(n_tosses)]
    return tosses.count("H") / n_tosses

for n in [10, 100, 1000, 10000]:
    print(f"{n} tosses -> P(Heads) approx. {simulate_coin(n):.3f}")
```

Результат:

```
10 tosses -> P(Heads) approx. 0.200
100 tosses -> P(Heads) approx. 0.580
1000 tosses -> P(Heads) approx. 0.496
10000 tosses -> P(Heads) approx. 0.500
```

По мірі того, як число підкидувань зростає → ймовірність стабілізується около 0.5.

6.2 Використання NumPy для роботи з розподілами ймовірностей.

6.2.1 Рівномірний розподіл.

В наступному прикладі генеруємо випадкові числа між 0 та 1, та виводимо ймовірність того, що значення із цього розподілу менше за 0.5.

```
import numpy as np

samples = np.random.rand(5)
print(samples)

p = np.mean(samples < 0.5)
print("P(X < 0.5) approx.", p)
```

Результат:

```
[0.75955599 0.02132349 0.45415367 0.32714836 0.93499247]
P(X < 0.5) approx. 0.6
```

6.2.2 Гаусівський розподіл.

Гаусівський розподіл (Normal, Gaussian Distribution) є розподілом по дійсних числах і, ймовірно, є найпоширенішим розподілом у науці про дані (я не кажу, що це найпоширеніший розподіл у самих даних, а в науці про дані як галузі). Це пояснюється багатьма приємними властивостями розподілу: він легко інтегрується, має природний багатовимірний аналог, а також тим фактом, про який ми поговоримо в декількох лекціях, що певні розподіли даних середніх значень вибірки мають тенденцію до гаусівського розподілу (так звана центральна гранична теорема).

Гаусівський розподіл має «дзвоноподібну» функцію щільності, яка, ймовірно, знайома багатьом людям.

Згенеруємо гаусівський розподіл із середнім 0 та відхиленням 1 засобами NumPy:

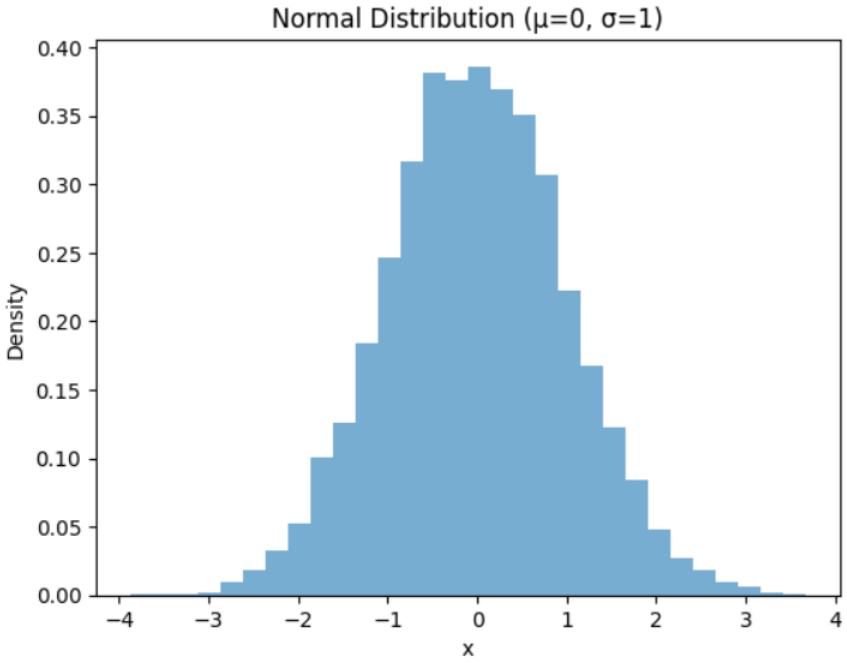
```
samples = np.random.normal(loc=0, scale=1, size=10000)
print("Mean:", np.mean(samples))
print("Std Dev:", np.std(samples))

import matplotlib.pyplot as plt

plt.hist(samples, bins=30, density=True, alpha=0.6)
plt.title("Normal Distribution (nu=0, sigma=1)")
plt.xlabel("x")
plt.ylabel("Density")
plt.show()
```

Результат:

```
Mean: -0.002846126267790618
Std Dev: 0.9992466721005764
```



6.2.3 Дискретні розподіли.

Задамо параметри розподілу: кількість випробувань 10 та ймовірність успіху 0.5. Обчислимо ймовірність випадання рівно 3 "решок"за 10 підкидань, та кумулятивну ймовірність $P(X \leq 3)$.

```
from scipy.stats import binom

n = 10
p = 0.5

prob = binom.pmf(k=3, n=n, p=p)
```

```
print(f"P(X=3) = {prob:.4f}")

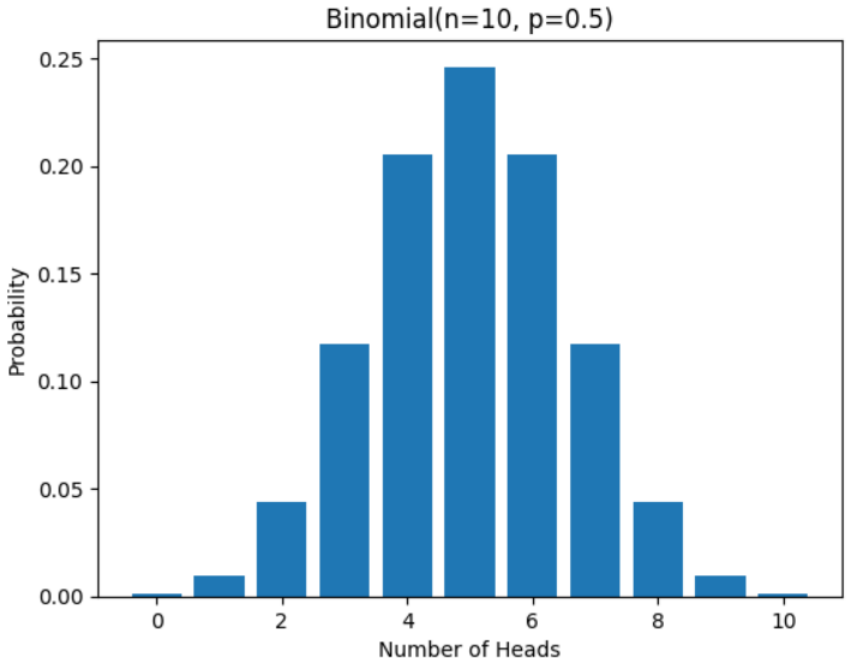
cum_prob = binom.cdf(k=3, n=n, p=p)
print(f"P(X <= 3) = {cum_prob:.4f}")

x = np.arange(0, 11)
y = binom.pmf(x, n, p)

plt.bar(x, y)
plt.xlabel("Number of Heads")
plt.ylabel("Probability")
plt.title("Binomial(n=10, p=0.5)")
plt.show()
```

Результат:

```
P(X=3) = 0.1172
P(X <= 3) = 0.1719
```



6.3 Теорема Байєса.

Обчислимо ймовірність захворювання за умови позитивного результату тесту. Якщо 1 % населення хворіє, то P(Захворювання) = 0,01. Точність тесту становить 99 %, то P(Позитивний результат | Захворювання) = 0,99; P(Позитивний результат | Відсутність захворювання) = 0,01. Обчислимо P(Захворювання | Позитивний результат):

```
P_D = 0.01
P_Pos_given_D = 0.99
P_Pos_given_notD = 0.01

P_Pos = P_Pos_given_D * P_D + P_Pos_given_notD * (1 - P_D)
```

```
P_D_given_Pos = (P_Pos_given_D * P_D) / P_Pos
print(f"P(Disease | Positive Test) = {P_D_given_Pos:.3f}")
```

Результат:

```
P(Disease | Positive Test) = 0.500
```

✎ **Питання для самоконтролю.**

1. Що вивчає теорія ймовірності?
2. Чому теорія ймовірності є важливою для інтелектуального аналізу даних?
3. Що таке випадковий експеримент?
4. Що таке елементарна подія?
5. Що називають простором елементарних подій?
6. Що таке випадкова подія?
7. Які види подій розрізняють у теорії ймовірності?
8. Що таке достовірна подія?
9. Що таке неможлива подія?
10. Які події називають несумісними?
11. Які події називають незалежними?
12. Як визначається ймовірність події?
13. Які властивості має ймовірність?
14. Як обчислюється ймовірність суми несумісних подій?
15. Як обчислюється ймовірність суми довільних подій?
16. Як обчислюється ймовірність перетину незалежних подій?
17. Що таке умовна ймовірність?
18. Як обчислюється умовна ймовірність?
19. У чому полягає правило множення ймовірностей?
20. Що таке повна група подій?
21. У чому полягає формула повної ймовірності?
22. Що таке випадкова величина?
23. Чим дискретна випадкова величина відрізняється від неперервної?
24. Що таке закон розподілу випадкової величини?
25. Що таке функція розподілу?
26. Що таке математичне сподівання?

27. Як обчислюється математичне сподівання дискретної випадкової величини?
28. Який практичний зміст має математичне сподівання?
29. Що таке дисперсія випадкової величини?
30. Для чого використовується стандартне відхилення?
31. Що показує дисперсія в аналізі даних?
32. У чому полягає закон великих чисел?
33. Яке практичне значення закону великих чисел?
34. Як закон великих чисел застосовується в машинному навчанні?
35. Що таке біноміальний розподіл?
36. У яких задачах застосовується біноміальний розподіл?
37. Що таке розподіл Бернуллі?
38. Чим розподіл Бернуллі відрізняється від біноміального?
39. Що таке нормальний розподіл?
40. Якими властивостями характеризується нормальний розподіл?
41. Чому нормальний розподіл часто зустрічається в реальних даних?
42. Що таке розподіл Пуассона?
43. У яких практичних задачах застосовується розподіл Пуассона?
44. У чому полягає теорема Байеса?
45. Як обчислюється апостеріорна ймовірність за теоремою Байеса?
46. Яке практичне значення має теорема Байеса для аналізу даних?
47. Як теорема Байеса використовується в задачах класифікації?
48. Що таке наївний байєсівський класифікатор і на якому припущенні він ґрунтується?
49. Яку роль відіграють ймовірнісні моделі в сучасному інтелектуальному аналізі даних?
50. Чому розуміння основ теорії ймовірності є необхідним для побудови моделей машинного навчання та прийняття рішень на основі даних?

Розділ 7

Вступ в машинне навчання.

Машинне навчання є на даний момент швидкозростаючою галуззю, хоча слід підкреслити, що початкові методи, які ми тут розглянемо (а саме лінійна регресія), з'явилися приблизно на 200 років раніше, ніж сам термін (зазвичай вважається, що Гаусс розробив метод найменших квадратів близько 1800 року). Більшість пізніших методів, про які ми говоримо, були добре вивчені в статистиці на початку століття. Ми все одно будемо використовувати загальний термін «машинне навчання» для позначення всіх цих методів, але важливо знати, що ці ідеї не виникли в спільноті ML. Натомість тема машинного навчання розвинулася завдяки поєднанню трьох різних елементів:

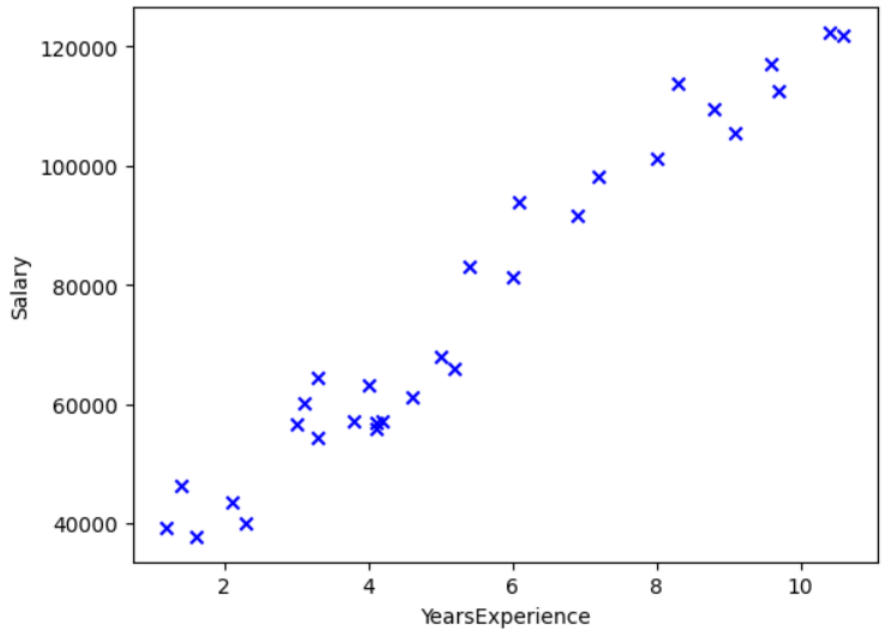
- зростання обчислювальної потужності (машинне навчання, яке виникло з інформатики, завжди було фундаментально пов'язане з обчислювальними алгоритмами),
- величезні обсяги доступних даних (сировина для методів машинного навчання) та
- деякі значні досягнення в галузі алгоритмів, що відбулися за останні 30 років. Однак для початку ми розглянемо деякі з найпростіших алгоритмів, щоб систематизувати основні принципи.

```
import matplotlib.pyplot as plt
import pandas as pd

def build_plot(file_path, x, y):
    data = pd.read_csv(file_path)
    plt.scatter(data[x], data[y], c='blue', marker='x')
    plt.xlabel(x)
    plt.ylabel(y)
    return plt

build_plot('../resources/Salary_dataset.csv', 'YearsExperience', 'Salary').show()
```

Результат:



Ось простий приклад залежності між стажем роботи співробітників та їхньою заробітною платою. Чи можна передбачити другий параметр на основі значення першого?

Можна припустити, що залежність відповідає лінійній моделі.

$$y \approx \theta_1 \cdot x + \theta_2$$

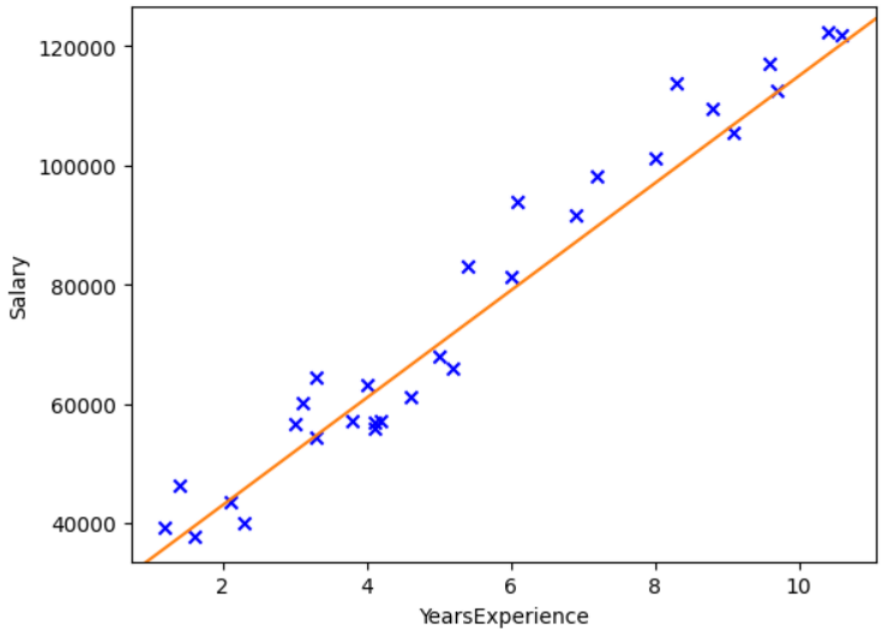
Тут - θ_1 це "нахил" лінії, та (slope) - θ_2 зсув. (intercept)

Чи можемо ми робити передбачення даних?

```
import numpy as np

def build_line(plt, theta):
    xlim, ylim =(plt.gca().get_xlim(), plt.gca().get_ylim())
    plt.plot(xlim, [theta[0]*xlim[0]+theta[1], theta[0]*xlim[1]+theta[1]], 'C1')
    plt.xlim(xlim)
    plt.ylim(ylim)

plt = build_plot('../resources/Salary_dataset.csv', 'YearsExperience', 'Salary')
build_line(plt, np.array([9000, 25000]))
```



7.1 Градієнтний спуск.

Припустимо, що прогнозована зарплата відповідає лінійній моделі. Як ми можемо знайти θ?

Є багато можливостей, але природною метою є мінімізація деякої різниці між цією лінією і спостережуваними даними, наприклад, квадратична втрата:

$$E(\theta) = \sum_i (\theta_1 * x_i + \theta_2 - y_i)^2$$

Тепер нам потрібно знайти параметри, які мінімізують значення функції $E(\theta)$.

$$minimize_\theta E(\theta)$$

Найкращий спосіб знайти значення функції $f(x)$, в якій її значення є мінімальним, — це знайти похідну. Ми починаємо з деякої точки і змінюємо вхідне значення в напрямку від'ємної похідної. У випадку багатозмінних функцій ми будемо використовувати часткові похідні.

Щоб знайти хороше значення θ, ми можемо повторно робити кроки в напрямку негативних похідних для кожного значення. Ітерація:

$$\theta_1 := \theta_1 - \alpha \frac{\delta}{\delta\theta_1} E(\theta_1, \theta_2)$$

$$\theta_2 := \theta_2 - \alpha \frac{\delta}{\delta\theta_2} E(\theta_1, \theta_2)$$

де α — це невелике додатне число, яке називається розміром кроку. Це алгоритм градієнтного спуску, основа сучасного машинного навчання.

Тут:

$$\frac{\delta}{\delta\theta_1} E(\theta_1, \theta_2) = \sum_i 2(\theta_1 * x_i + \theta_2 - y_i) * x_i$$

$$\frac{\delta}{\delta\theta_2} E(\theta_1, \theta_2) = \sum_i 2(\theta_1 * x_i + \theta_2 - y_i)$$

Застосуємо градієнтний спуск (Gradient descent) до наших даних, спочатку нормалізувавши їх.

```
def build_plot_normalized(file_path, x, y):
    data = pd.read_csv(file_path)
    x_nor = (data[x] - min(data[x])) / (max(data[x]) - min(data[x]))
    y_nor = (data[y] - min(data[y])) / (max(data[y]) - min(data[y]))
    plt.scatter(x_nor, y_nor, c='blue', marker='x')
    plt.xlabel(x)
    plt.ylabel(y)
    return plt

build_plot_normalized('../resources/Salary_dataset.csv', 'YearsExperience',
    'Salary').show()
```

Візуалізуємо процес градієнтного спуску. Поки що, проведемо лише одну ітерацію.

```
# Visualizing the gradient descent:

def get_data(file_path, x_name, y_name):
    data = pd.read_csv(file_path)
    return data[x_name], data[y_name]

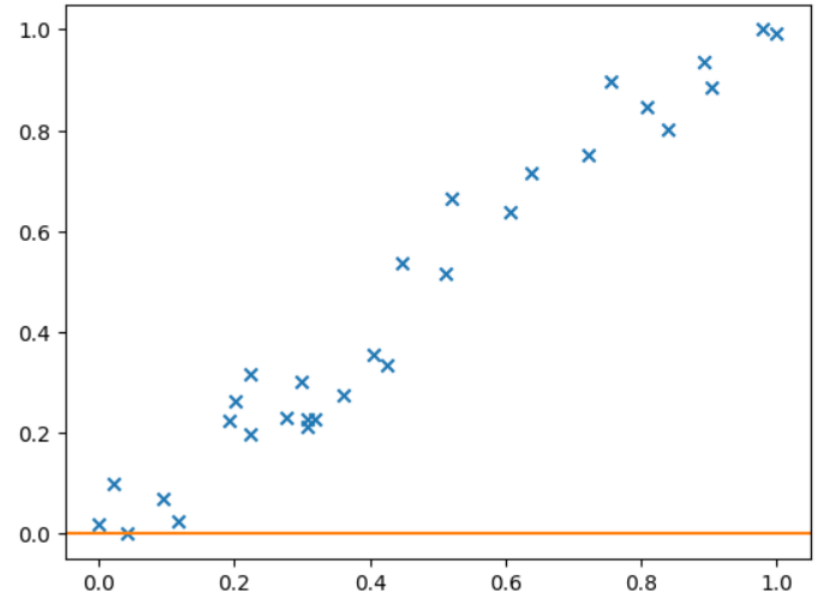
def normalize_data(x):
    x_normalized = (x - min(x)) / (max(x) - min(x))
    return x_normalized

def gradient_descent(x, y, iters, alpha):
    theta = np.array([0., 0.])
    for t in range(iters):
        theta[0] -= alpha/len(x) * 2 * sum((theta[0] * x + theta[1] - y)*x)
        theta[1] -= alpha/len(x) * 2 * sum((theta[0] * x + theta[1] - y) )
    return theta

def plot_fit(x, y, theta):
    plt.scatter(x, y, marker = 'x')
    xlim, ylim =(plt.gca().get_xlim(), plt.gca().get_ylim())
    plt.plot(xlim, [theta[0]*xlim[0]+theta[1], theta[0]*xlim[1]+theta[1]], 'C1')
    plt.xlim(xlim)
    plt.ylim(ylim)

x, y = get_data('../resources/Salary_dataset.csv', 'YearsExperience', 'Salary')
x_normalized, y_normalized = normalize_data(x), normalize_data(y)

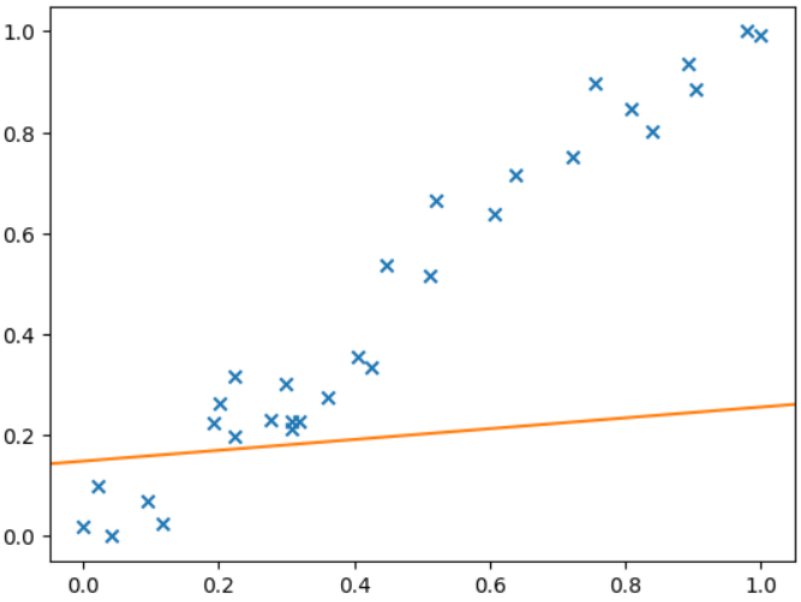
#first iteration
theta = gradient_descent(x_normalized, y_normalized, 0, 0.1)
plot_fit(x_normalized, y_normalized, theta)
```



Як бачимо, на першій ітерації значення параметрів θ віддалені від бажаних. Проведемо іще декілька ітерацій.

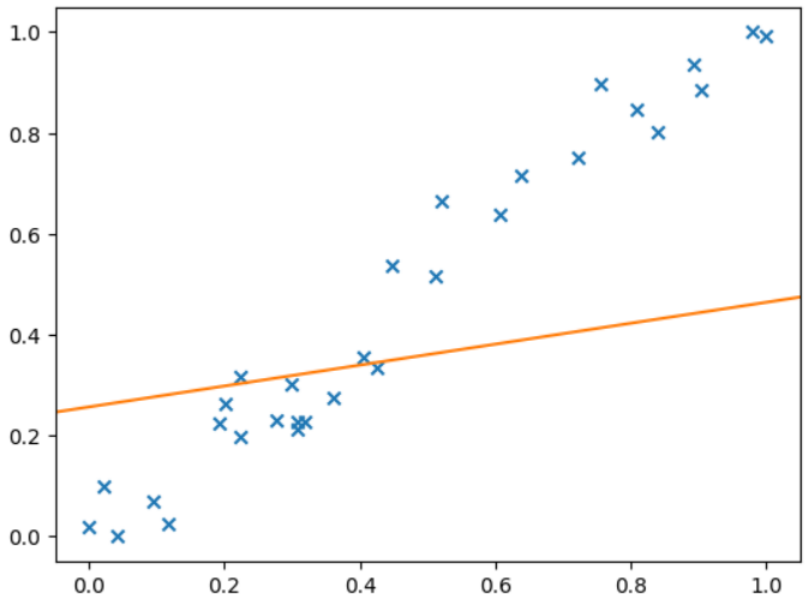
```
theta = gradient_descent(x_normalized, y_normalized, 2, 0.1) # change third
    parameter to see more iterations
plot_fit(x_normalized, y_normalized, theta)
```

Результат:



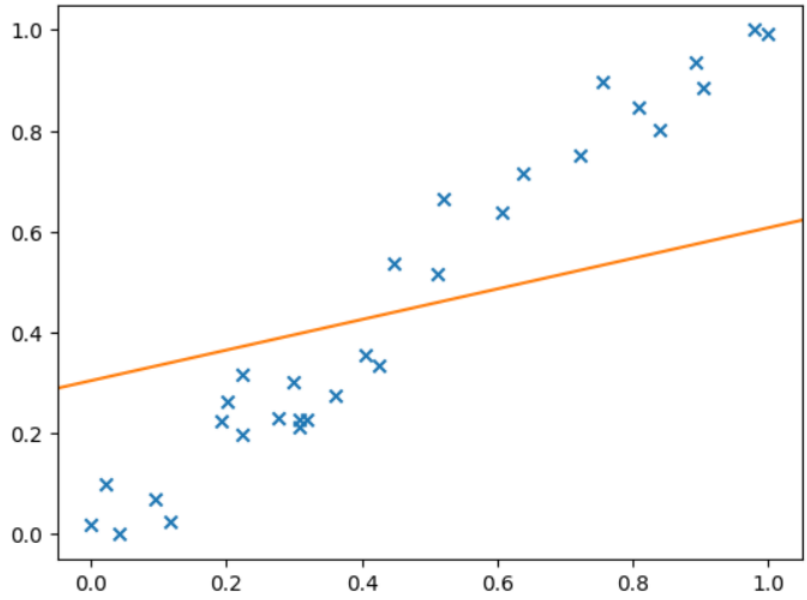

```
theta = gradient_descent(x_normalized, y_normalized, 5, 0.1) # change third
    parameter to see more iterations
plot_fit(x_normalized, y_normalized, theta)
```

Результат:



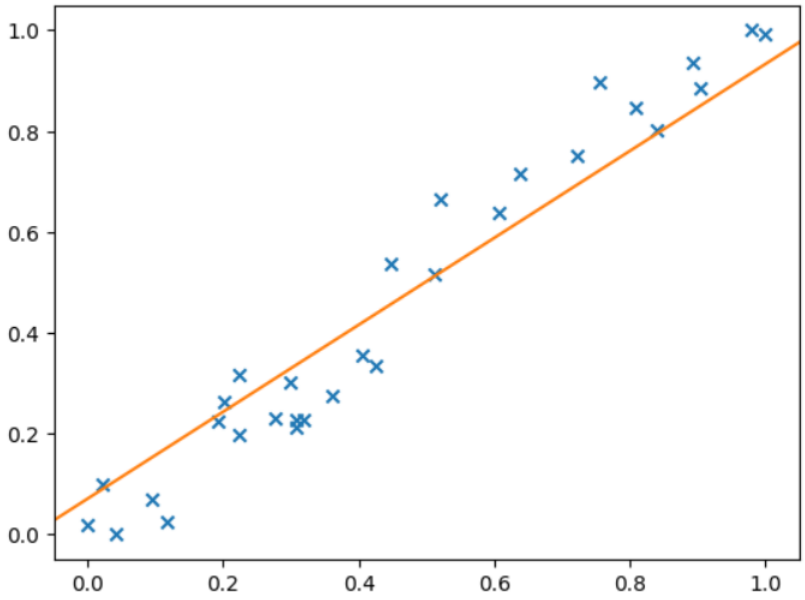

```
theta = gradient_descent(x_normalized, y_normalized, 10, 0.1) # change third
    parameter to see more iterations
plot_fit(x_normalized, y_normalized, theta)
```

Результат:




```
theta = gradient_descent(x_normalized, y_normalized, 100, 0.1) # change third
    parameter to see more iterations
plot_fit(x_normalized, y_normalized, theta)
```

Результат:



Після 100-ї ітерації ми наблизилися із нашою лінійною моделлю до реального розподілу. Ми також можемо візуалізувати процес зміни параметрів θ:

```
# Visualizing parameter updates

def gradient_descent_params(x, y, iters):
    thetas = []
    theta = np.array([0., 0.])
    alpha = 1.0
    for t in range(iters):
        thetas.append(theta.copy())
        theta[0] -= alpha/len(x) * 2 * sum((theta[0] * x + theta[1] - y)*x)
        theta[1] -= alpha/len(x) * 2 * sum((theta[0] * x + theta[1] - y) )
    return np.array(thetas)

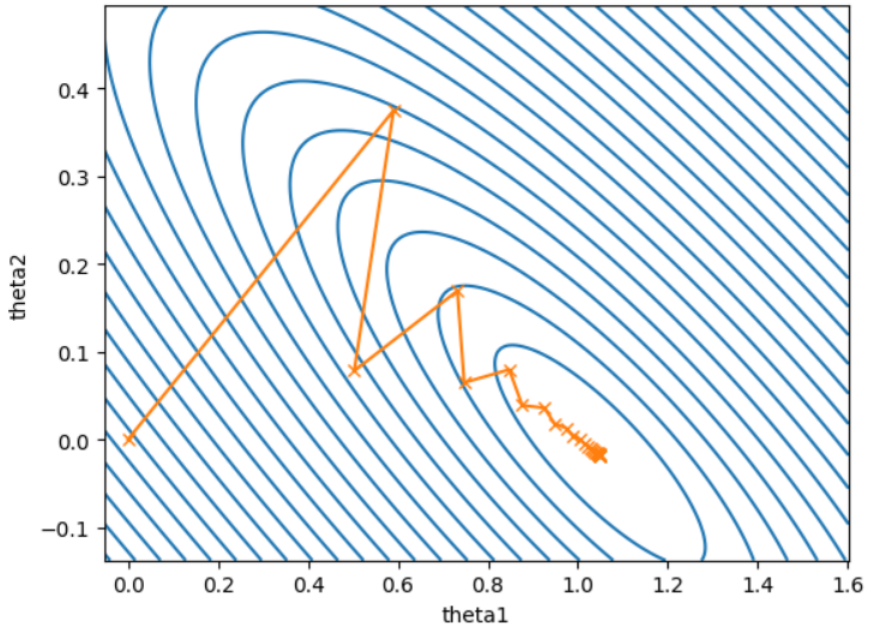
def err(x, y, theta):
    return np.mean((np.outer(x, theta[:,0]) + theta[:,1] - y[:,None])**2,axis=0)

thetas = gradient_descent_params(x_normalized, y_normalized, 100)
plt.plot(thetas[:,0], thetas[:,1])
xlim, ylim =(np.array(plt.gca().get_xlim()), np.array(plt.gca().get_ylim()))
xlim += np.array([0,0.5])
ylim += np.array([-0.1, 0.1])

XX,YY = np.meshgrid(np.linspace(xlim[0],xlim[1],200), np.linspace(ylim[0],
    ylim[1],200))
ZZ = err(x_normalized.to_numpy(), y_normalized.to_numpy(),
    np.hstack([np.ravel(XX)[:,None], np.ravel(YY)[:,None]])).reshape(XX.shape)
#V = np.logspace(np.log(np.min(ZZ)), np.log(np.max(ZZ)), 30)
V = np.linspace(np.sqrt(np.min(ZZ)), np.sqrt(np.max(ZZ)), 25)**2
plt.clf()
plt.contour(XX,YY,ZZ, V, colors=('C0',))
plt.plot(thetas[:,0], thetas[:,1], 'C1-x')
```

```
plt.xlabel("theta1")
plt.ylabel("theta2")
```

Результат:



7.1.1 Бібліотека scikit-learn.

Зробимо те саме лінійне прогнозування, використовуючи бібліотеку scikit-learn. Для того, щоб використати модель лінійної регресії, її спочатку потрібно імпортувати:

```
from sklearn.linear_model import LinearRegression
```

Перетворимо наші дані в масиви numpy (саме з цими масивами і працює scikit-learn) та перетворимо вектор даних у матрицю за допомогою методу `.reshape()`

```
data = pd.read_csv('../resources/Salary_dataset.csv')
X = np.array(data['YearsExperience']).reshape(-1, 1)
y = np.array(data['Salary'])
```

☞ Більшість алгоритмів бібліотеки scikit-learn очікують, що вхідні дані матимуть вигляд `(кількість_об'єктів, кількість_ознак)`. Тобто, навіть якщо ознака лише одна, модель все одно хоче отримати матрицю, а не звичайний вектор. Саме тому і варто застосовувати метод `.reshape()`.

Етап тренування моделі:

```
model = LinearRegression()
model.fit(X, y)
```

Ми можемо отримати параметри theta:

```
slope = model.coef_[0]     # theta_1
intercept = model.intercept_ # theta_2
print("slope:", slope, "intercept:", intercept)
```

Та, зрештою, побудуємо лінію:

```
X_line = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
y_line = model.predict(X_line)

plt.scatter(X, y, c='blue', marker='x')
plt.plot(X_line, y_line) # line
plt.xlabel("X")
plt.ylabel("y")
plt.title("Linear Regression Fit")
plt.show()
```

Весь код разом:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

data = pd.read_csv('../resources/Salary_dataset.csv')
X = np.array(data['YearsExperience']).reshape(-1, 1)
y = np.array(data['Salary'])

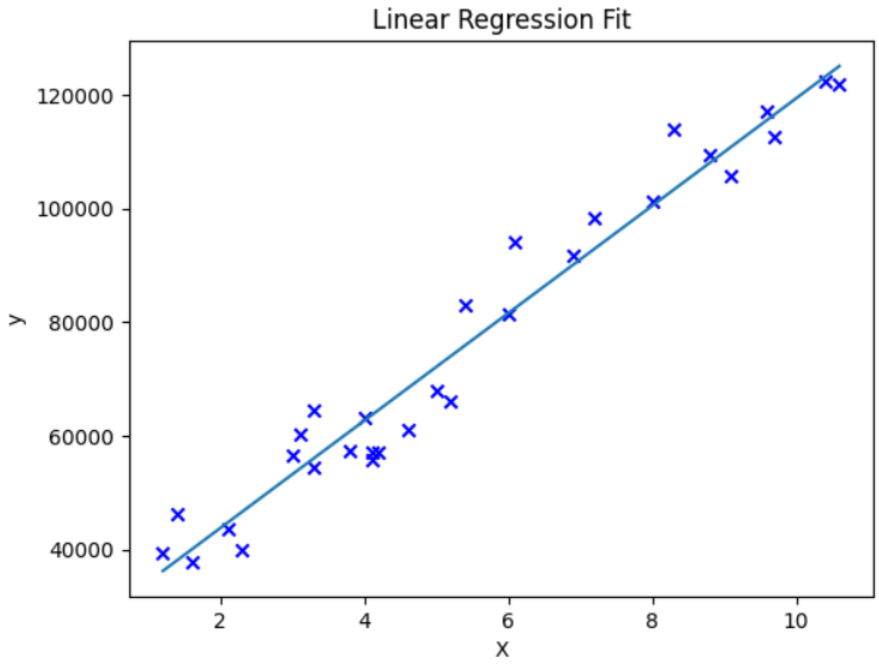
model = LinearRegression()
model.fit(X, y)

slope = model.coef_[0]    # theta_1
intercept = model.intercept_ # theta_2
print("slope:", slope, "intercept:", intercept)

X_line = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
y_line = model.predict(X_line)

# plot
plt.scatter(X, y, c='blue', marker='x')
plt.plot(X_line, y_line) # line
plt.xlabel("X")
plt.ylabel("y")
plt.title("Linear Regression Fit")
plt.show()
```

Результат:



7.2 Трохи більше про машинне навчання.

Машинне навчання надає можливість автоматично встановлювати прогнозну модель на основі даних.

Термінологія. Функція втрат: $l : y \times y \to \mathrm{R}$, вимірює різницю між прогнозом і фактичним результатом

$$l(y', y) = (y' - y)^2$$

Канонічна задача оптимізації машинного навчання:

$$minimize_\theta \sum_i l(h_\theta(x^i), y^i)$$

Практично кожен алгоритм машинного навчання має таку форму, просто вкажіть

- Що таке функція гіпотези?
- Що таке функція втрат?
- Як вирішити задачу оптимізації?

7.2.1 Приклади алгоритмів машинного навчання.

- Найменші квадрати: лінійна гіпотеза, квадратична втрата, (зазвичай) аналітичне рішення.
- Лінійна регресія: лінійна гіпотеза.
- Машина опорних векторів (support vector machine): лінійна або ядрова гіпотеза, втрата шарніра, etc.
- Нейронна мережа: складена нелінійна функція, etc., (зазвичай) градієнтний спуск)
- Дерево рішень (decision trees): ієрархічні півплощини, вирівняні по осі, жадібна оптимізація

- Наївний Байєс: лінійна гіпотеза, спільна ймовірність за певних припущень незалежності, аналітичне рішення.

✎ **Питання для самоконтролю.**

1. Що таке машинне навчання?
2. Чим машинне навчання відрізняється від традиційного програмування?
3. Які основні етапи побудови моделі машинного навчання?
4. Що таке навчальна вибірка?
5. Для чого використовується тестова вибірка?
6. Що таке ознака (feature)?
7. Що таке цільова змінна (target)?
8. Які основні типи задач машинного навчання існують?
9. Що таке навчання з учителем (Supervised Learning)?
10. Що таке навчання без учителя (Unsupervised Learning)?
11. Що таке навчання з підкріпленням (Reinforcement Learning)?
12. Чим задача класифікації відрізняється від задачі регресії?
13. Наведіть приклади задач класифікації.
14. Наведіть приклади задач регресії.
15. Що таке кластеризація?
16. Які практичні задачі можна розв'язувати за допомогою машинного навчання?
17. Що таке модель машинного навчання?
18. Що означає навчити модель?
19. Що таке прогнозування (prediction)?
20. Що таке лінійна регресія?
21. Для яких задач використовується лінійна регресія?
22. Який вигляд має рівняння простої лінійної регресії?
23. Що означає коефіцієнт нахилу (slope) у рівнянні лінійної регресії?
24. Що означає вільний член (intercept)?
25. Як інтерпретувати коефіцієнти лінійної регресії?
26. Що таке прогнозоване значення моделі?
27. Що таке похибка прогнозування?
28. Чому модель не завжди проходить через усі точки навчальної вибірки?
29. Що таке функція втрат (Loss Function)?

30. Для чого використовується середньоквадратична помилка (MSE)?
31. Як обчислюється середньоквадратична помилка?
32. Чому необхідно мінімізувати функцію втрат?
33. Що таке оптимізація моделі?
34. Що таке градієнтний спуск?
35. Яка основна ідея алгоритму градієнтного спуску?
36. Що таке градієнт функції?
37. Чому під час оптимізації рухаються у напрямку, протилежному градієнту?
38. Що таке швидкість навчання (learning rate)?
39. Як вибір швидкості навчання впливає на процес навчання моделі?
40. Які проблеми можуть виникнути при надто великому значенні learning rate?
41. Які проблеми можуть виникнути при надто малому значенні learning rate?
42. Що таке ітерація градієнтного спуску?
43. Коли алгоритм градієнтного спуску завершує свою роботу?
44. Чим Batch Gradient Descent відрізняється від Stochastic Gradient Descent?
45. Що таке Mini-Batch Gradient Descent?
46. Які переваги використання бібліотеки Scikit-learn для побудови моделей машинного навчання?
47. Яку роль відіграє бібліотека NumPy у задачах машинного навчання?
48. Як оцінити якість побудованої моделі лінійної регресії?
49. Чому важливо розділяти дані на навчальну та тестову вибірки?
50. Які обмеження має лінійна регресія та в яких випадках доцільно використовувати складніші моделі машинного навчання?

Розділ 8

Лінійна Класифікація.

Розділ 9

Нелінійні Алгоритми Машинного Навчання.

Розділ 10

Ансамблеве навчання.

Розділ 11

Навчання без учителя.

Розділ 12

Перевірка статистичних гіпотез.

Розділ 13

Рекомендаційні Системи.

Розділ 14

Часові Ряди.

Розділ 15

Моделювання Кризових Явищ.

Розділ 16

Використання нейронних мереж для аналізу даних.